

HealthCloud: Consent-Aware Care Coordination & Claims Platform

Final Source-of-Truth, System Design, and Implementation Plan

Status	Finalized and Approved for Implementation
Document Version	1.0
Generation Date	July 25, 2026

This document is the authoritative design and implementation plan for HealthCloud. The complete project design has been finalized. No project-level implementation-blocking decisions remain and no Needs Verification items remain. "Approved for implementation" means the design is ready to build; it does not mean that the application has already been implemented, deployed to real healthcare users, certified, or operated with real patient data.

Document-use principle: project-level decisions are frozen; implementation-time details may evolve only when they do not contradict this source-of-truth.

Table of Contents

Part I - Document Control and Project Foundation	8
1. Document Status and Purpose	8
2. Final Project Identity and Positioning	9
3. Problem Statement	10
4. Why the Project Is Being Built	11
Part II - Scope, Boundaries, and Product Decisions	13
5. Final Project Boundary	13
6. Final Recommended Project Scope	15
7. Healthcare and Compliance Boundaries	16
8. Final MVP and Advanced Scope Matrix	17
Part III - Users, Organizations, and Journeys	19
9. Users and Roles	19
10. Organization and Multi-Tenant Model	20
11. Final User Journeys	22
Part IV - Functional Modules	25
12. Core Features	25
13. Additional Approved Features and Future Scope	25
14. Patient and Care-Coordination Module	26
15. Clinical Context Module	28
16. Claims Intake and Support Workflow	29
17. Synthetic Claims-Adjudication Engine	31
18. Advanced Claims Processing	32
19. Documents and Communication	34

Part V - Identity, Authorization, Privacy, and Governance	36
20. Authentication Design	36
21. Authorization Architecture	37
22. Consent Management	38
23. Field-Level Data Visibility and Masking	39
24. Emergency Break-Glass Access	40
25. Security Considerations and Threat Model	41
26. Secure Logging Rules	43
27. Audit Logging and Explainability	44
28. Data Retention, Deletion, and Legal History	45
Part VI - Frontend, Backend, Data, APIs, and Architecture	47
29. Exact Technology Stack	47
30. Frontend Design and Architecture	48
31. Backend Design and Architecture	51
32. Database Design	53
33. Detailed Claims Data Model	56
34. API Design	56
35. Error-Handling Model	58
36. Total Project Architecture	59
37. High-Level and Low-Level Design	60
38. Status and Workflow State Machines	61
Part VII - Production Engineering	63
39. Advanced Production Features	63
40. Event-Driven Architecture	63
41. Search, Reporting, and Exports	65

42. Notifications	65
Part VIII - Cloud, Delivery, Testing, and Operations	67
43. AWS Cloud Deployment	67
44. Environment Strategy	67
45. CI/CD Plan	68
46. Testing Strategy	69
47. Observability and Monitoring	71
48. Backup and Disaster Recovery	72
49. Operational Runbooks	73
50. Cloud-Cost Plan	74
Part IX - Design-Before-Coding Deliverables	75
51. Meaning of Phase 0	75
52. Architecture Decision Records	76
53. Product-Planning Documents	76
Part X - Decision Finalization and Implementation Strategy	78
54. Finalizing the Design versus Implementing in Phases	78
55. Category A Decisions - Finalized Before Coding	78
56. Category B Decisions - Finalized for Their Phases	79
Part XI - Final Implementation Plan	80
57. Reconciliation of the Phase Plans	80
58. Final Main Implementation Phases	80
Phase 0 - Project Design and Planning	80
Phase 1 - Application Foundation and Multi-Tenant Identity	81
Phase 2 - Core Care-Coordination Workflow	82
Phase 3 - Consent, Authorization, Privacy and Documents	83

Phase 4 - Clinical Context and Basic Claims Intake	83
Phase 5 - Basic Synthetic Claims-Adjudication Engine	84
Phase 6 - Advanced Claims Processing	84
Phase 7 - Advanced Healthcare Security and Governance	85
Phase 8 - Event-Driven Production Features	85
Phase 9 - Search, Reporting and Enterprise Usability	86
Phase 10 - Cloud Deployment and CI/CD	86
Phase 11 - Observability, Recovery and Operations	86
Phase 12 - Final Validation and Portfolio Presentation	87
59. Final Implementation Sequence	87
60. MVP Acceptance Criteria	88
Part XII - Real-World Engineering and Portfolio Presentation	90
61. Real-World Engineering Depth	90
62. Engineering Decisions and Trade-Offs	90
63. Repository Structure	90
64. GitHub README Direction	91
65. Architecture and Design Diagrams	91
66. Resume and Interview Value	92
67. LinkedIn Post Plan	93
68. Demo Video Direction	93
69. Future Improvements	94
Part XIII - Verification, Conflicts, and Traceability	95
70. Decision History	95
71. Rejected or Replaced Ideas	95

72. Resolved Decisions	96
73. Final Verification Notes	96
74. Complete Heading Traceability Matrix	96
75. Completeness and Consistency Audit	98
76. Information Preservation Report	99
Points of Content Need	101
Current Finalization Status	102
Final Audit Result	104
Appendix A - Architecture Decision Records	105
A.1 ADR-001 - Modular Monolith First	105
A.2 ADR-002 - Shared PostgreSQL Multi-Tenancy	105
A.3 ADR-003 - Backend-Derived Tenant Context	105
A.4 ADR-004 - Cognito + Spring BFF	105
A.5 ADR-005 - Hybrid Authorization	105
A.6 ADR-006 - Limited Clinical Context	105
A.7 ADR-007 - Bounded Synthetic Claims Engine	105
A.8 ADR-008 - PostgreSQL Metadata + Private S3 Documents	106
A.9 ADR-009 - Transactional Outbox + Kafka	106
A.10 ADR-010 - Per-Organization HMAC Audit Chains	106
A.11 ADR-011 - Concurrency Strategy	106
A.12 ADR-012 - ECS Fargate Target Deployment	106
A.13 ADR-013 - Terraform + GitHub Actions	106
A.14 ADR-014 - Immutable Consent and Claim Decisions	106
A.15 ADR-015 - On-Demand Full AWS Validation	106
A.16 ADR-016 - Single-Origin Browser Architecture	106

A.17 ADR-017 - PostgreSQL First for Cache/Search	106
Appendix B - Decision History and Replaced Decisions	107
Appendix C - Complete Heading Traceability Matrix	108
Appendix D - Completeness and Consistency Audit	109
Appendix E - Information Preservation Report	110
Appendix F - Final Project Audit and Implementation Readiness	111
F.1 Readiness summary	111
F.2 Project-level audit counters	111
F.3 Implementation readiness statement	111

Part I - Document Control and Project Foundation

1. Document Status and Purpose

1.1 Project and document identity

Field	Final value
Project	HealthCloud: Consent-Aware Care Coordination & Claims Platform
Document	Final Source-of-Truth, System Design, and Implementation Plan
Status	Finalized and Approved for Implementation
Version	1.0
Lifecycle state	Final design; implementation has not yet begun
Intended audience	Project owner, software engineers, architects, security reviewers, QA, DevOps, hiring managers, recruiters, and technical interviewers

1.2 Purpose

This document defines the complete approved HealthCloud product boundary, business workflows, users and organizations, security and privacy model, care-coordination behavior, limited clinical context, synthetic claims processing, documents, event-driven architecture, database and API design, frontend and backend architecture, AWS deployment, CI/CD, testing, observability, recovery, operational practices, implementation phases, and portfolio presentation strategy.

It is intended to prevent architectural drift during implementation. Later implementation details may be refined only when they do not change an approved project-level decision. Examples include class names, final indexes after query-plan analysis, UI spacing, exact CloudWatch thresholds after baseline measurement, and internal Terraform module decomposition.

1.3 Status terminology

Term	Meaning
Finalized	Approved project requirement or architectural policy; implementation must follow it unless the design is deliberately revised later.
MVP	First stable end-to-end functional milestone, completed through Phase 5 after Phase 0 design.
Advanced	Approved project capability implemented after the functional MVP.
Future enhancement	Intentionally excluded from the approved build roadmap and retained only as a possible later evolution.
Recommendation	Documentation or implementation guidance that does not change a finalized product requirement.
Rejected/replaced	Earlier idea superseded by the final decision.
Implementation-time detail	Detail intentionally deferred because it depends on code, testing, or measurement rather than project design.

1.4 Limitations and truthfulness rules

- HealthCloud is a design and portfolio implementation plan, not an already completed production platform.
- The project uses synthetic healthcare data only.
- The platform must not be described as a certified HIPAA-compliant commercial product.

- No production traffic, uptime, response-time, scale, security, recovery, test-coverage, or cost result may be claimed until it is actually measured.
- No employer, client, or real patient information may be copied into the project.
- No confidential workplace architecture, schemas, documents, or implementation details may be reused.
- Numerical performance, security, and operational evidence will be added only after controlled testing.

2. Final Project Identity and Positioning

2.1 Final project name

HealthCloud: Consent-Aware Care Coordination & Claims Platform

2.2 Meaning of the name

- HealthCloud communicates a healthcare-inspired, cloud-native platform.
- Consent-Aware communicates the primary technical differentiator: access depends on consent, role, organization, relationship, purpose of use, business need, and field-level visibility.
- Care Coordination & Claims Platform identifies the two principal business workflows that the system connects.

2.3 One-sentence description

HealthCloud is a multi-tenant, consent-aware care coordination and synthetic claims platform for multiple healthcare organizations. All organizations use the same platform, while their users, patients, consents, requests, clinical summaries, claims, documents, configuration, and audit history remain securely isolated.

2.4 Complete project overview

HealthCloud is a healthcare-inspired full-stack system that manages patient-related service requests, care coordination, limited clinical context, documents, consent, claims intake, synthetic claims adjudication, appeals, reprocessing, auditability, and production-style operations.

The system supports multiple fictional healthcare organizations in one shared application and PostgreSQL database. Tenant-owned data is associated with an organization and protected by backend-enforced tenant context, object relationships, consent, purpose of use, business need, field-level policy, and audit requirements.

The application supports patients, providers, care coordinators, claims reviewers, organization administrators, platform administrators, and auditors. Emergency break-glass access is a temporary elevated access mode available only to approved clinical/coordination roles; it is not a permanent unrestricted role.

2.5 Core engineering story

A healthcare-inspired workflow platform where access is controlled by identity, organization, role, patient relationship, consent, purpose of use, business need, and field classification - and every sensitive action is auditable.

2.6 Most important positioning correction

HealthCloud is not a simple patient/provider dashboard. Dashboards are only interfaces into a backend system whose engineering value comes from:

- Multi-tenant isolation.
- Hybrid role- and attribute-based authorization.

- Patient/provider/care-team relationships.
- Consent and purpose-of-use evaluation.
- Field-level visibility and masking.
- Controlled workflow state machines.
- Explainable synthetic claims adjudication.
- Immutable/versioned decisions and histories.
- Secure document processing.
- Transactional outbox and Kafka.
- Reliability controls such as retry, idempotency, DLQ, replay, and optimistic locking.
- Cloud deployment, CI/CD, observability, backup, recovery, runbooks, cost engineering, and measurable evidence.

2.7 Final healthcare and compliance wording

Use the following positioning:

HealthCloud is a healthcare-inspired, HIPAA-aligned engineering portfolio platform that uses synthetic data to demonstrate privacy-aware authorization, auditability, secure document handling, tenant isolation, and production-style engineering practices.

Do not claim that HealthCloud is certified HIPAA compliant, approved for production healthcare use, or used with real PHI.

2.8 Synthetic-data positioning

HealthCloud contains only synthetic/fake healthcare data. No real patient, employer, client, or production healthcare data is used.

Base synthetic patient demographics and healthcare histories are generated with the open-source Synthea patient simulator. HealthCloud transforms only the data needed by the Limited Clinical Context module and enriches it with deterministic project-specific seed data for organizations, facilities, users, assignments, consents, service requests, plans, claims, prior authorizations, referrals, appeals, documents, audit records, and security scenarios.

Synthetic diagnosis data uses a curated ICD-10-CM FY2026 subset. Synthetic service/procedure data uses a curated HCPCS Level II July 2026 public-use subset. CPT content is not bundled.

The demo dataset is deterministic so the same core scenarios can be recreated for tests and demonstrations.

3. Problem Statement

3.1 Business problem

Healthcare-style workflows involve many participants who require different information for different reasons. Patients, providers, coordinators, claims reviewers, administrators, and auditors should not receive the same data or the same actions. Requests also require ownership, assignment, status control, supporting documents, timelines, claim decisions, and accountability.

3.2 Technical problem

The platform must coordinate identity, tenant isolation, patient relationships, consent, purpose, workflow state, claims calculations, documents, field masking, audit history, and asynchronous processing without scattering critical rules across controllers or trusting frontend behavior.

3.3 Privacy and access problem

Authentication answers who the user is. It does not answer whether the user:

- belongs to the correct healthcare organization;
- is assigned to the patient;
- has a legitimate care, claim, or review relationship;
- has a permitted purpose of use;
- is covered by an active patient consent directive;
- needs a specific field for the current business action;
- is allowed to perform the requested state transition;
- should be denied and audited.

3.4 Care-coordination problem

Patient-related requests must be submitted, triaged, assigned, reviewed, sent back for information, approved/rejected, cancelled when permitted, closed, and fully traced. Multiple users may attempt updates concurrently, so status transitions must be validated and protected from silent overwrites.

3.5 Claims-processing problem

A realistic claim is not one CRUD row. It contains a header, multiple line items, eligibility, plan benefits, deductibles, copays, coinsurance, allowed amounts, benefit limits, exclusions, documentation requirements, provider-network status, prior authorization, referral requirements, human review, anomaly signals, appeals, reprocessing, and decision history.

HealthCloud implements a bounded synthetic claims engine rather than pretending to reproduce every rule of a real insurance platform.

3.6 Auditability problem

Sensitive reads and writes, authorization denials, consent decisions, role changes, assignments, documents, claims decisions, manual overrides, exports, and break-glass sessions must be attributable, searchable, and resistant to silent alteration.

3.7 Why RBAC alone is insufficient

A PROVIDER role cannot by itself determine whether a provider may view patient X. HealthCloud therefore combines broad role permissions with attributes and relationships:

```
Authentication
-> Tenant
-> Function permission
-> Object relationship
-> Consent
-> Purpose of use
-> Business need
-> Field visibility
-> ALLOW / DENY
-> Audit
```

4. Why the Project Is Being Built

4.1 Engineering objective

The goal is to demonstrate the ability to design and build a production-style enterprise application rather than a tutorial CRUD project. HealthCloud is a public, non-confidential system that reinforces Java/Spring Boot, React/TypeScript, PostgreSQL, Kafka, AWS, security, and healthcare-domain experience.

4.2 Real-world engineering value

The project demonstrates:

- system and domain design;
- modular backend architecture;
- relational database design;
- multi-tenancy;
- authentication and session management;
- hybrid authorization;
- consent and minimum-necessary access;
- secure document handling;
- transactional consistency and concurrency;
- rules-driven financial calculation;
- explainable decisions and human review;
- event-driven reliability;
- cloud deployment and infrastructure as code;
- CI/CD and security scanning;
- testing, observability, recovery, and operational runbooks;
- cost-aware architectural decisions.

4.3 Portfolio, resume, and interview value

HealthCloud is intended to be the flagship portfolio project. After implementation it should support concise, defensible resume bullets and deep interview conversations about trade-offs, failures, evidence, and design decisions. The project is particularly relevant to SWE II, Backend Engineer, Full-Stack Engineer, Java Developer, Cloud Engineer, platform-oriented application roles, and healthcare software roles.

It should make a hiring manager conclude that the engineer can think beyond feature tickets and reason about secure systems in production.

4.4 What makes it more than CRUD

HealthCloud includes conditional access decisions, multi-step state machines, rules-based calculations, versioned histories, asynchronous events, failure recovery, multi-tenant boundaries, secure object storage, audit integrity, observability, and measurable engineering evidence.

Part II - Scope, Boundaries, and Product Decisions

5. Final Project Boundary

5.1 What HealthCloud is

A synthetic-data, multi-tenant care-coordination and claims platform that demonstrates secure identity and access, consent, limited clinical context, controlled request workflows, claims intake, explainable synthetic adjudication, human review, documents, auditability, events, cloud delivery, monitoring, and recovery.

5.2 What HealthCloud is not

HealthCloud is not:

- a complete EHR;
- a medication-management, laboratory, imaging, prescribing, inpatient, or hospital-management system;
- a real insurer's complete adjudication and payment platform;
- a real provider-contract management platform;
- a commercial subscription-billing SaaS;
- a certified HIPAA product;
- a repository of real PHI;
- a copy of a workplace system.

5.3 MVP

After Phase 0 design, the functional MVP is completed through Phase 5:

1. Application foundation and multi-tenant identity.
2. Core care-coordination workflow.
3. Consent, authorization, privacy, and documents.
4. Limited clinical context and basic claims intake.
5. Basic synthetic claims-adjudication engine.

5.4 Advanced approved scope

Phases 6-11 add:

- provider network, prior authorization, referral checks;
- anomaly detection;
- manual override and human review;
- appeals and reprocessing;
- break-glass access;
- tamper-evident audit history and access reviews;
- transactional outbox and Kafka;
- search, reporting, exports, and accessibility;

- AWS deployment and CI/CD;
- observability, backup, restore, cost controls, and runbooks.

5.5 Future improvements

Approved future-only directions include:

- FHIR-style resource modeling and broader interoperability;
- more advanced consent interoperability and granularity;
- enterprise database-per-tenant isolation where justified;
- cross-organization patient-sharing policies;
- more advanced audit and anomaly analytics;
- application-level field encryption for a real sensitive-data deployment;
- RLS as defense in depth if the operational model justifies it;
- integration with separate portfolio platform tools.

5.6 Explicit exclusions and constraints

- Real patient data and employer data are prohibited.
- Full EHR scope is excluded.
- Real claim settlement/payment disbursement is excluded.
- Family deductibles, coordination of benefits, secondary insurance, out-of-pocket maximum, and balance billing are not part of the initial synthetic engine.
- Redis, Elasticsearch/OpenSearch, and a database-per-tenant model are not introduced without measured need.
- Microservices are not the starting architecture.
- SMS, push, WhatsApp, and voice notifications are excluded.
- Public patient self-registration is excluded.
- The project must remain implementable and defensible by one engineer.

5.7 Definition of done

The complete project is done only when:

- main workflows operate end to end;
- cross-tenant and object-level denials are proven;
- consent and field-level visibility work;
- request and claim state machines reject invalid transitions;
- claim decisions and calculations are explainable;
- documents are securely uploaded, scanned, and authorized;
- audit history is complete and tamper-evident;
- event processing demonstrates outbox, retry, idempotency, DLQ, and replay;
- AWS deployment is reproducible;
- CI/CD and security scanning pass;
- dashboards, alerts, backup/restore, and runbooks are demonstrated;

- documentation, evidence, demo video, and portfolio materials are complete.

6. Final Recommended Project Scope

6.1 Identity and organization

- Multiple healthcare organizations.
- First-class but lightweight facilities.
- One shared application and PostgreSQL database.
- Backend-enforced organization context.
- Organization membership and role management.
- Organization administrator and platform administrator separation.
- One active organization membership per normal user in the initial version.
- Each patient belongs to exactly one organization in the initial version.

6.2 Patient and care coordination

- Patient profiles.
- Provider profiles.
- Provider-patient assignments.
- Care-coordinator assignments.
- Service requests, assignment, comments, priority, due dates, timelines, cancellation, and closure.
- Role-specific dashboards.
- SLA and escalation support.

6.3 Consent and authorization

- Versioned consent directives.
- Consent GRANT and DENY effects.
- Purpose, data category, provider/care-team/organization scope, effective time, expiration, revocation, and supersession.
- Hybrid RBAC and attribute-based authorization.
- Tenant, object, relationship, consent, purpose, business need, and field-level enforcement.
- Secure disclosure behavior and denied-access auditing.

6.4 Clinical context

- Diagnosis summaries.
- Treatment-plan summaries.
- Encounter summaries.
- Related provider and facility where applicable.
- Service dates.
- Supporting documents.
- Claim links.

- No complete EHR.

6.5 Claims and adjudication

- Claim header and line items.
- Eligibility, plan, benefit rules, deductibles, copays, coinsurance, limits, and exclusions.
- Provider-network status.
- Prior authorization and referral requirements.
- Line-level outcomes and claim-level partial approval.
- Manual review, anomaly signals, appeals, and reprocessing.
- Immutable/versioned adjudication history.

6.6 Documents and communication

- Private S3 object storage.
- PostgreSQL document metadata.
- Direct-to-S3 uploads.
- File type, size, content signature, malware scan, quarantine, and authorized downloads.
- Comments, timelines, in-app notifications, and privacy-preserving email.

6.7 Platform operations

- Docker and local development profile.
- Terraform AWS infrastructure.
- GitHub Actions CI/CD.
- CodeQL, Dependabot, Trivy, and OWASP ZAP.
- CloudWatch, OpenTelemetry, X-Ray, dashboards, alerts, backups, restore exercises, runbooks, and cost evidence.

6.8 Final decision

The complete approved design is retained and implemented in controlled phases. No project-level design item remains unresolved or pending verification.

7. Healthcare and Compliance Boundaries

7.1 Approved compliance language

HealthCloud may be described as a healthcare-inspired, HIPAA-aligned engineering demonstration. It must not be marketed as HIPAA certified or as an operational healthcare product.

7.2 Clinical-data boundary

The Limited Clinical Context module exists only to support care coordination, consent/access demonstrations, and synthetic claims.

Included:

- diagnosis summary;
- treatment-plan summary;

- encounter summary;
- service date;
- related provider/facility;
- synthetic diagnosis and service codes;
- supporting document references;
- claim links.

Excluded:

- full longitudinal clinical chart;
- medication-management platform;
- detailed notes system;
- laboratory/imaging platform;
- prescribing;
- real clinical decision support.

7.3 Clinical code policy

Synthetic diagnosis values use a curated ICD-10-CM FY2026 subset. Synthetic service/procedure values use a curated HCPCS Level II July 2026 subset. The schema stores `code_system`, `code`, `display_name`, `code_version`, `effective_from`, and `effective_to` instead of hard-coding a single coding system into column names.

7.4 Synthetic-data pipeline

```
Synthea
-> synthetic patient histories
-> HealthCloud transformation/seed process
-> only approved Limited Clinical Context fields
-> deterministic project-specific workflows and claims
-> PostgreSQL / S3
```

The seed dataset deliberately contains scenarios for allowed access, cross-tenant denial, unassigned-provider denial, active/revoked/expired consent, approved/denied/partially approved/pended claims, prior authorization, referral, appeal, anomaly, break-glass, and audit review.

8. Final MVP and Advanced Scope Matrix

Capability	MVP	Advanced	Future / Excluded	Rationale
Multi-tenant organizations	Yes	Governance refinements	DB-per-tenant future	Core isolation proof
Facilities	Lightweight	Reporting/config refinement	Full facility management excluded	Healthcare realism without hospital scope
Patient/provider profiles and assignments	Yes	Access review	Cross-org sharing future	Core object authorization
Service requests	Yes	SLA/event automation	-	Core care workflow
Consent and field masking	Yes	Governance analytics	More interoperability future	Core differentiator
Secure documents	Yes	Event-driven scan/retention ops	-	Workflow evidence
Limited clinical context	Yes	-	Full EHR excluded	Claims/care support only
Claims intake	Yes	-	Real insurer breadth excluded	Core claim flow
Basic synthetic adjudication	Yes	-	Real payment settlement excluded	Explainable backend depth

Capability	MVP	Advanced	Future / Excluded	Rationale
Network/prior auth/referrals	-	Phase 6	-	Requires stable basic claims
Appeals/reprocessing	-	Phase 6	-	Requires immutable decisions
Break-glass/tamper-evident audit	-	Phase 7	-	Advanced security/governance
Kafka/outbox	-	Phase 8	-	Added after stable transactions
Search/reports/exports/accessibility	Incremental	Phase 9 completion	-	Enterprise usability
AWS/CI/CD	Local foundation	Phase 10	-	Production-style proof
Observability/recovery/runbooks	Basic health early	Phase 11	Multi-region DR excluded	Operational proof
Permanent 24/7 full AWS stack	No	On-demand validation	Expensive idle services destroyed	Cost discipline
Real patient data	No	No	Explicitly excluded	Privacy and safety

Part III - Users, Organizations, and Journeys

9. Users and Roles

9.1 Patient

- Synthetic patient represented by a patient profile in one organization.
- Can view own permitted profile, requests, consent history, documents, and patient-facing claim outcomes.
- Can create supported service requests, grant/revoke consent, supply information, and upload authorized documents.
- Cannot view another patient's resources, internal audit data, staff administration, or unrelated claims.
- MFA is mandatory.
- Sensitive profile and document actions are audited.

9.2 Provider

- Healthcare professional associated with an organization and optional facility memberships.
- Can view assigned patients and assigned/related requests only when tenant, relationship, consent, purpose, and field policies pass.
- Can add permitted clinical summaries, comments, responses, and supporting documents.
- Cannot access unassigned patients or another organization's data.
- MFA is mandatory.

9.3 Care coordinator

- Organization-scoped staff member responsible for triage, assignment, follow-up, due dates, and workflow coordination.
- Receives only minimum-necessary patient/provider information.
- Can assign requests and coordinate workflow according to the state machine.
- Cannot see another organization's data or unrestricted clinical/claims content.
- MFA is mandatory.

9.4 Claims reviewer

- Organization-scoped staff member assigned to claim validation, manual review, adjudication review, appeals, and supporting evidence.
- Can see claims, line items, coverage, eligibility, approved supporting clinical fields, rule results, and documents relevant to the claim.
- Cannot use the claims role as unrestricted access to clinical data.
- MFA is mandatory.

9.5 Organization administrator

- Manages users, roles, facilities, assignments, organization configuration, workflow configuration, and organization-scoped audit information.
- Cannot access another organization.
- Does not receive unrestricted patient-data browsing merely because of the administrator role.

- May hold `PRIVACY_REVIEW` permission.
- MFA is mandatory.

9.6 Platform administrator

- Manages tenant lifecycle, platform health, and global technical configuration.
- Does not casually browse tenant patient/claim content.
- Exceptional technical access requires explicit reason and high-priority audit handling.
- Platform-only events use a separate platform audit chain.
- MFA is mandatory.

9.7 Auditor

- Read-only oversight role for audit metadata, authorization decisions, workflow/claims histories, exports, and emergency-access review.
- Normally sees metadata rather than medical content.
- Can hold `PRIVACY_REVIEW` permission.
- Auditor searches/exports are themselves auditable.
- MFA is mandatory.

9.8 Break-glass user

Break-glass is a temporary emergency authorization mode, not a permanent role. Only Provider and Care Coordinator may initiate it. Same-organization membership, active account, normal authorization failure, mandatory reason, and fresh MFA are required.

9.9 Privacy review permission

A separate permanent privacy-reviewer role is not created. Instead, `PRIVACY_REVIEW` permission may be assigned to selected auditors and organization administrators. The person who initiates break-glass cannot review their own event.

10. Organization and Multi-Tenant Model

10.1 Final tenant model

HealthCloud uses:

- one shared application;
- one shared PostgreSQL database;
- an `organization_id` tenant key on tenant-owned records;
- backend-derived tenant context;
- database relationship constraints preventing cross-tenant references;
- authorization and negative testing.

10.2 Organization ownership

Normal users have exactly one active organization membership in the initial project. Patients belong to exactly one organization. Cross-organization patient exchange and multi-organization normal-user sessions are future capabilities, not current scope.

10.3 Facilities

facility is a lightweight first-class entity with fields such as:

```
facility_id
organization_id
name
facility_type
synthetic_address
status
created_at
updated_at
```

facility_membership associates providers/staff with facilities. Patients remain organization-owned rather than permanently owned by a facility. Encounters, requests, claims, assignments, and prior authorizations may reference a facility when relevant.

10.4 Authenticated organization context

After login the backend resolves a request-scoped user context similar to:

```
userId
organizationId
roles
permissions
```

The browser may display organization information but cannot choose an arbitrary organization identifier to bypass tenant isolation.

10.5 Tenant-safe querying

A typical query condition is conceptually:

```
SELECT *
FROM patient
WHERE id = :patientId
AND organization_id = :currentOrganizationId;
```

Important tenant relationships also use composite organization-aware constraints where practical, for example a claim's (organization_id, patient_id) references the patient's matching organization and identifier.

10.6 Why not database-per-tenant initially

Database-per-tenant provides stronger physical isolation but would add unnecessary operational complexity to this portfolio implementation: multiple connections, migrations, provisioning, backup plans, monitoring, and higher cost. Shared-database isolation is the final initial design. Database-per-tenant remains a future enterprise option.

10.7 Demo organizations

Two fictional organizations are used to prove isolation:

- NorthCare Health
- Green Valley Medical Group

Earlier example dataset sizes may be used for seeding but are demonstration data, not scale claims.

10.8 Demonstration isolation cases

- Assigned NorthCare provider -> permitted NorthCare patient fields when all other policies pass.
- Unassigned NorthCare provider -> denied.

- NorthCare user -> Green Valley patient -> denied.
- Green Valley administrator -> NorthCare users/audit -> denied.
- Platform administrator -> platform health without ordinary tenant clinical content.

11. Final User Journeys

11.1 Patient journey

1. Patient activates an administrator-provisioned account and completes MFA enrollment.
2. Patient authenticates through Cognito and receives a HealthCloud server-side session.
3. Patient opens the own-patient dashboard.
4. Patient reviews or creates consent directives for approved purposes/data categories.
5. Patient creates a service request such as claim support, document review, appointment help, referral request, or benefit clarification.
6. Backend derives organization and validates patient ownership.
7. Patient optionally requests secure document upload authorization and uploads a synthetic document directly to private S3.
8. Request enters DRAFT or SUBMITTED according to the action.
9. Patient receives in-app and privacy-preserving email notifications.
10. When NEEDS_INFORMATION is returned, the patient supplies additional information.
11. Patient may cancel in approved pre-decision states.
12. Patient views the final request/claim-support outcome and timeline.

Denied path: attempts to access another patient's resource use secure disclosure behavior and are audited.

11.2 Care-coordinator journey

1. Coordinator authenticates with MFA.
2. Opens organization-scoped request queue.
3. Backend applies tenant and role filters.
4. Triage SUBMITTED requests.
5. Reviews minimum-necessary patient/provider context.
6. Sets priority and due date.
7. Assigns to provider/reviewer.
8. Advances only through valid transitions.
9. Adds comments and follow-up information.
10. SLA reminders/escalations are generated asynchronously after Phase 8.
11. Closes requests only after approved/rejected completion conditions.

Cross-tenant access, invalid transitions, and prohibited fields are denied and audited.

11.3 Provider journey

1. Provider authenticates with MFA.

2. Opens assigned patient/request list.
3. Backend checks tenant, provider-patient relationship, consent, purpose, business need, and field policy.
4. Permitted limited clinical context is returned.
5. Provider adds an authorized summary, response, comment, or document.
6. Request timeline and audit history are updated.

Unassigned patient, expired/revoked consent, wrong purpose, cross-tenant access, or disallowed fields cause denial.

11.4 Claims-reviewer journey

1. Reviewer authenticates with MFA.
2. Opens organization claims queue.
3. Backend confirms role, tenant, assignment/business need, and allowed clinical fields.
4. Reviewer examines claim header, lines, coverage, eligibility, documents, and rule results.
5. Missing information may cause the claim to pend for review.
6. Reviewer performs manual review and may request information.
7. Any override requires an explicit reason.
8. Final decision or reprocessing result is versioned and audited.

11.5 Auditor journey

1. Auditor authenticates with MFA.
2. Searches audit records within authorized scope.
3. Medical content remains masked/omitted unless separately authorized.
4. Filters by actor, organization, patient reference, resource, action, result, reason, time, or emergency indicator.
5. Reviews authorization, consent, claim, workflow, and break-glass evidence.
6. Exports only authorized audit reports.
7. Auditor searches/exports create audit records.

11.6 Organization-administrator journey

1. Administrator authenticates with MFA.
2. Manages organization users, roles, facilities, assignments, configuration, and selected audit views.
3. Cross-organization targets are rejected.
4. Sensitive administrative changes are audited.
5. If assigned `PRIVACY_REVIEW`, administrator reviews break-glass events unless they initiated the event.

11.7 Platform-administrator journey

1. Platform administrator authenticates with MFA.
2. Views organization lifecycle and platform health.
3. Creates/disables organizations and manages global technical settings.
4. Normal platform operations exclude tenant clinical content.
5. Exceptional incident investigation requires a reason and high-priority audit event.

11.8 Break-glass journey

1. Provider or care coordinator encounters a normal authorization denial during a genuine emergency.
2. User initiates break-glass for a patient in the same organization.
3. User provides mandatory justification.
4. Fresh reauthentication and fresh MFA succeed.
5. A single active user + patient emergency session is created for 15 minutes; no extension is supported.
6. Access is limited to patient identity/contact data, limited clinical context, relevant encounter/treatment summaries, and active care information.
7. Claims financial history, unrelated documents, audit records, bulk export, and other patients remain unavailable unless separately permitted.
8. Organization administrator and a `PRIVACY_REVIEW` holder are notified immediately.
9. High-priority audit event records actor, patient, organization, reason, resources, start, expiry, and later review details.
10. Session automatically expires after 15 minutes.
11. Review must be completed within 24 hours by a different person.
12. Suspicious usage triggers security investigation.

Part IV - Functional Modules

12. Core Features

Feature	Final behavior	Classification
Role-specific accounts	Cognito identity plus HealthCloud organization/role context and mandatory MFA	MVP
Patient profiles	Organization-owned synthetic patient data with object authorization	MVP
Provider-patient assignments	Effective relationship controlling provider access	MVP
Consent management	Versioned GRANT/DENY directives with purpose, scope, dates, and history	MVP
Service-request management	Create, triage, assign, review, request information, approve/reject, cancel, close	MVP
Controlled transitions	Backend state machine plus optimistic locking	MVP
Claim-support workflow	Patient/provider/coordinator/reviewer collaboration with minimum necessary access	MVP
Document management	Private S3, authorized direct upload/download, scan/quarantine, metadata	MVP
Comments and timeline	Request-scoped communication and immutable status/action history	MVP
Audit logging	Sensitive allowed/denied reads and writes plus administrative/security activity	MVP
Notifications	In-app and privacy-preserving email	Advanced Phase 8
Role-specific dashboards	Patient, provider, coordinator, claims, admin, platform, auditor	MVP/Advanced refinement

12.1 Feature design rules

For every protected feature:

- Tenant context is derived by the backend.
- Function permission is checked before business execution.
- Object relationship is checked where relevant.
- Consent, purpose, and business need are evaluated for sensitive patient data.
- Field-level policy shapes the response.
- State transitions are validated in the backend.
- Sensitive allowed and denied actions are audited.
- Frontend hiding/disablement improves UX but does not enforce security.

13. Additional Approved Features and Future Scope

13.1 Approved advanced features

- Transactional outbox.

- Kafka publishing/consuming.
- Idempotent consumers.
- Retry/backoff and dead-letter handling.
- Event versioning and controlled replay.
- Break-glass access.
- Tamper-evident per-organization audit chains.
- Retention controls and access review.
- Search, reporting, exports, and accessibility.
- Backup/restore exercises.
- Cost evidence and budget controls.
- Formal Evidence Pack.

13.2 Future enhancements

- FHIR-style resource modeling and APIs.
- Broader consent interoperability/versioning.
- Cross-organization data sharing.
- Database-per-tenant enterprise deployment.
- Dedicated privacy-officer role if future organization complexity requires it.
- Application-level field encryption for real sensitive-data deployments.
- More advanced anomaly and audit analytics.
- Specialized cache/search services only after measurement.

14. Patient and Care-Coordination Module

14.1 Patient onboarding and account provisioning

Public self-registration is not supported. An organization administrator provisions the patient/user, Cognito sends an invitation/activation flow, the patient completes account activation and TOTP MFA, and HealthCloud links the identity-provider subject to the internal user and patient profile.

This intentionally avoids solving identity proofing for arbitrary public registrants.

14.2 Patient profile

Core synthetic profile information includes organization ownership, member/reference identifier, permitted demographics/contact fields, and relationships to requests, consent, clinical summaries, documents, and claims.

14.3 Provider-patient and coordinator relationships

Assignments are organization-scoped, effective-dated, versioned, auditable, and have states:

PENDING
ACTIVE
EXPIRED
REVOKED

Providers require an active patient relationship for ordinary clinical access. Coordinators receive relationship/assignment access appropriate to care coordination.

14.4 Service-request types

```
CLAIM_SUPPORT
REFERRAL_REQUEST
DOCUMENT_REVIEW
APPOINTMENT_HELP
BENEFIT_CLARIFICATION
```

14.5 Request features

- create and submit;
- triage;
- assign/reassign;
- priority and due date;
- comments;
- supporting documents;
- information requests;
- controlled approval/rejection;
- cancellation when permitted;
- closure;
- timeline/history;
- search/filtering after Phase 9;
- SLA reminders/escalations after Phase 8.

14.6 Final service-request states

```
DRAFT
SUBMITTED
TRIAGED
ASSIGNED
UNDER_REVIEW
NEEDS_INFORMATION
APPROVED
REJECTED
CANCELLED
CLOSED
```

Principal flow:

```
DRAFT -> SUBMITTED -> TRIAGED -> ASSIGNED -> UNDER_REVIEW
                                         -> NEEDS_INFORMATION -> UNDER_REVIEW
                                         -> APPROVED -> CLOSED
                                         -> REJECTED -> CLOSED
```

Cancellation:

```
DRAFT -> CANCELLED
SUBMITTED -> CANCELLED
TRIAGED -> CANCELLED
ASSIGNED -> CANCELLED
NEEDS_INFORMATION -> CANCELLED
```

Patient may cancel DRAFT, SUBMITTED, or NEEDS_INFORMATION before final decision. Authorized coordinator may cancel SUBMITTED, TRIAGED, ASSIGNED, or NEEDS_INFORMATION with a reason. Provider has no general cancellation authority. CANCELLED and CLOSED are terminal in the initial implementation; reopening requires creating a new request.

Every transition records from-state, to-state, actor, role, organization, reason, timestamp, version, and correlation

identifier.

14.7 Care plans, tasks, referrals, encounters, and follow-up

- Treatment-plan summaries are part of Limited Clinical Context.
- Encounters are represented as summaries rather than full clinical episodes.
- Referrals exist as a service-request use case and as a claims-support requirement.
- Follow-ups are represented through requests, comments, due dates, and SLA events.
- A separate full care-plan/task-management product is not created unless needed by an approved workflow.

14.8 Role-specific dashboards

- Patient: own requests, consent, documents, claim-support outcomes.
- Provider: assigned patients and requests.
- Coordinator: triage/assignment queue, priority, due dates, SLA status.
- Claims reviewer: claim queue, adjudication/manual review.
- Organization administrator: users, facilities, assignments, organization audit/configuration.
- Platform administrator: tenant/platform health and lifecycle.
- Auditor: audit search, break-glass review, decision history.

15. Clinical Context Module

15.1 Purpose

Provide only the clinical information necessary for care coordination and synthetic claims, while deliberately avoiding a full EHR.

15.2 Data included

- diagnosis summary;
- treatment-plan summary;
- encounter summary;
- related provider and optional facility;
- service dates;
- synthetic ICD-10-CM diagnosis codes;
- synthetic HCPCS Level II service/procedure codes where appropriate;
- supporting document references;
- claim links.

15.3 Access policy

Clinical context requires same-organization membership, appropriate role/function, patient relationship or approved claim/review relationship, applicable consent, approved purpose of use, business need, and field-level policy.

Claims reviewers receive only claim-relevant clinical context. Auditors normally see metadata rather than clinical content. Platform administrators do not receive clinical content for routine platform operations.

15.4 Audit

Sensitive reads, updates, links, and denied attempts are audited. Clinical content is never duplicated into ordinary operational logs or Kafka payloads.

15.5 Implementation phase

Phase 4.

16. Claims Intake and Support Workflow

16.1 Claim header

A claim contains:

- claim identifier;
- organization;
- patient/member;
- provider;
- insurance plan;
- optional facility;
- claim type;
- service period;
- submission time;
- total billed amount;
- workflow status;
- version.

16.2 Claim line items

Each line contains:

- service/procedure code and code system/version;
- diagnosis code(s) where applicable;
- quantity;
- billed amount;
- allowed amount;
- line-level outcome;
- denial/pend reason where applicable;
- patient and plan responsibility after adjudication.

16.3 Supporting records

- plan and eligibility;
- benefit-rule-set version;
- deductible and coverage-limit accumulators;

- provider-network status;
- prior authorization;
- referral;
- supporting documents;
- limited clinical summaries;
- manual review and appeal records.

16.4 Validation

Before adjudication:

- required header fields must be present;
- at least one line item must exist;
- organization relationships must be valid;
- patient/provider/plan references must be tenant-consistent;
- service dates must be valid;
- applicable eligibility/coverage must be resolvable;
- required documents, referral, and prior authorization data are evaluated;
- duplicate/idempotency controls are applied.

16.5 Claim workflow status versus decision outcome

Claim processing state and adjudication outcome are intentionally separate.

Final claim workflow statuses:

```
DRAFT
SUBMITTED
VALIDATING
READY_FOR_ADJUDICATION
ADJUDICATING
PENDED_FOR_REVIEW
UNDER_MANUAL_REVIEW
FINALIZED
UNDER_APPEAL
REPROCESSING
```

Final adjudication outcomes:

```
APPROVED
PARTIALLY_APPROVED
DENIED
PENDED_FOR_MANUAL_REVIEW
```

A claim can therefore be FINALIZED while its latest adjudication outcome is PARTIALLY_APPROVED.

16.6 Main claim workflow

Automated path:

```
DRAFT
-> SUBMITTED
-> VALIDATING
-> READY_FOR_ADJUDICATION
-> ADJUDICATING
-> FINALIZED
```

Manual review path:

```
ADJUDICATING
-> PENDING_FOR_REVIEW
-> UNDER_MANUAL_REVIEW
-> FINALIZED
```

or, after new information:

```
UNDER_MANUAL_REVIEW
-> READY_FOR_ADJUDICATION
-> ADJUDICATING
```

Appeal/reprocessing path:

```
FINALIZED
-> UNDER_APPEAL
-> REPROCESSING
-> READY_FOR_ADJUDICATION
-> ADJUDICATING
-> FINALIZED
```

Only finalized adjudications change financial accumulators.

17. Synthetic Claims-Adjudication Engine

17.1 Final boundary

HealthCloud implements a configurable, bounded Synthetic Claims-Adjudication Engine. It is designed to demonstrate realistic rule evaluation and explainability without claiming the breadth of a real insurer's platform.

17.2 Rule order

```
Validate claim
-> Check eligibility
-> Check coverage
-> Determine allowed amount
-> Apply coverage limit
-> Apply deductible
-> Apply copay
-> Apply coinsurance
-> Check exclusions
-> Check required documents
-> Check network / prior authorization / referral where applicable
-> Evaluate anomaly rules
-> Produce outcome or manual-review pend
```

17.3 Financial types and rounding

- Currency: USD only.
- Java money type: `BigDecimal`.
- PostgreSQL: `NUMERIC(19,2)`.
- Never use binary floating-point for monetary values.
- Rounding mode: `HALF_UP` at line-calculation boundaries.
- Claim totals equal the sum of finalized line results.

17.4 Basic line calculation

For a covered line:

```
allowed = min(billed, configuredAllowedAmount)
```

```

coveredAllowed = min(allowed, remainingCoverageLimit)
deductibleApplied = min(coveredAllowed, remainingDeductible)
remainingAfterDeductible = coveredAllowed - deductibleApplied
copayApplied = min(configuredCopay, remainingAfterDeductible)
coinsuranceBase = remainingAfterDeductible - copayApplied
coinsuranceApplied = round(coinsuranceBase * patientCoinsuranceRate)
patientResponsibility = deductibleApplied + copayApplied + coinsuranceApplied
planResponsibility = coveredAllowed - patientResponsibility

```

For out-of-network services, the plan uses a configured allowed amount or percentage. Balance billing is not modeled.

17.5 Outcomes and explanations

Every adjudication records:

- outcome;
- rule-set version;
- evaluated rules and pass/fail results;
- billed/allowed values;
- deductible, copay, coinsurance, patient responsibility, plan responsibility;
- reason codes;
- human-readable explanation;
- required next step;
- version and timestamps.

The system must be able to answer: what rules ran, what passed/failed, how the money was calculated, and why the final outcome occurred.

17.6 Rule-set versioning

Benefit/adjudication rules are immutable and effective-dated. Each decision records `rule_set_version_id`. The applicable version is selected using the date of service, not the current date, so historical decisions can be reproduced and explained.

17.7 Accumulators

Initial plan year is January 1-December 31. Track `deductible_used` and `coverage_limit_used`.

Approved/finalized adjudication updates accumulators transactionally. Reprocessing applies only the delta between the new finalized amount and the previous finalized amount, avoiding double counting.

17.8 Simplifications

The initial synthetic engine does not implement family deductibles, coordination of benefits, secondary insurance, out-of-pocket maximum, balance billing, full provider-contract pricing, actual payment settlement, or comprehensive commercial insurer rule libraries.

18. Advanced Claims Processing

18.1 Provider-network status

```

IN_NETWORK
OUT_OF_NETWORK
UNKNOWN

```

Network status can influence allowed amount, cost sharing, coverage, or manual review according to the synthetic rule set.

18.2 Prior authorization

Final status values:

```
PENDING
APPROVED
DENIED
CANCELLED
EXPIRED
```

The engine checks whether authorization is required, exists, was valid on the service date, and covers the submitted service/provider/facility as configured.

18.3 Referral rules

The claims engine can evaluate whether a referral is required, missing, expired, or mismatched to the provider/service.

18.4 Partial approval

Line-level outcomes:

```
APPROVED
DENIED
PENDED
```

Claim-level outcome derivation:

```
All lines APPROVED -> APPROVED
All lines DENIED -> DENIED
Any unresolved PENDED line -> PENDED_FOR_MANUAL_REVIEW
Mixture of APPROVED + DENIED, with no PENDED -> PARTIALLY_APPROVED
```

Only approved lines contribute to allowed amount and patient/plan responsibility. Denied lines have zero plan responsibility and require reason codes. An appeal may target the whole claim or individual denied lines.

18.5 Anomaly detection

Supported synthetic signals include duplicate claim, impossible service date, unusually high amount, provider-specialty mismatch, conflicting services, and abnormal provider submission volume.

Severity:

```
LOW -> record flag
MEDIUM -> reviewer warning
HIGH -> PENDED_FOR_MANUAL_REVIEW
```

An anomaly is never automatically labelled fraud and does not automatically deny a claim.

18.6 Manual review

Manual-review workflow states:

```
PENDING
IN_REVIEW
NEEDS_INFORMATION
RESOLVED
CANCELLED
```

Resolution:

```

APPROVE
DENY
OVERRIDE

```

Any override requires a reason and creates an audit event.

18.7 Appeals and reprocessing

Appeal states:

```

SUBMITTED
UNDER_REVIEW
NEEDS_INFORMATION
REPROCESSING
RESOLVED
WITHDRAWN

```

Resolution:

```

UPHELD
OVERTURNED
PARTIALLY_OVERTURNED

```

Rules:

- never overwrite the original adjudication;
- create a new adjudication version during reprocessing;
- preserve appeal evidence and reviewer actions;
- update financial accumulators only by the finalized difference;
- audit every appeal, override, and reprocessing action.

19. Documents and Communication

19.1 Storage model

Binary documents live in a private S3 bucket. Metadata and authorization relationships live in PostgreSQL. Direct browser uploads do not make S3 URLs an authorization mechanism; every upload/download authorization is issued after backend policy checks.

19.2 Allowed file policy

Allowed formats:

```

PDF
PNG
JPEG

```

Maximum size: 10 MB per file.

Reject ZIP/RAR archives, executables, scripts, macro-enabled documents, and unknown binary formats.

Validate extension, declared Content-Type, magic bytes/file signature, and file size.

19.3 Upload flow

```

User requests upload authorization
-> backend checks tenant/role/relationship/consent/purpose/file metadata
-> backend creates document metadata and short-lived upload authorization
-> browser uploads directly to private S3

```

```
-> document enters scan workflow  
-> file becomes AVAILABLE only after clean result
```

19.4 Document states

```
PENDING_UPLOAD  
UPLOADED  
SCANNING  
AVAILABLE  
QUARANTINED  
SCAN_FAILED  
REJECTED  
DELETED
```

19.5 Malware scanning

Local development uses a `MalwareScanner` abstraction with a deterministic fake implementation capable of `CLEAN`, `THREAT_FOUND`, and `SCAN_FAILED` results.

AWS staging/demo uses GuardDuty Malware Protection for S3. Files remain unavailable unless a clean result is confirmed. Threat/failed scans remain quarantined or blocked.

19.6 Download authorization

Downloads require the same tenant, object, relationship, consent, purpose, and field/content policies as the related workflow. Short-lived authorized links are issued only after a successful decision.

19.7 Communication

Notifications are limited to `IN_APP` and `EMAIL`.

Email contains no detailed health or claim information. A safe message is: "Your HealthCloud request has been updated. Sign in to view the details." Detailed diagnoses, claim contents, and document data are never sent in email notifications.

Local email development uses Mailpit. AWS demonstration uses Amazon SES.

Part V - Identity, Authorization, Privacy, and Governance

20. Authentication Design

20.1 Final authentication architecture

HealthCloud uses Amazon Cognito User Pool + OIDC/OAuth 2.0 Authorization Code Grant + Spring Boot Backend-for-Frontend (BFF) + server-side session.

Earlier browser-held JWT designs are replaced and are not active requirements.

20.2 Account provisioning

There is no public patient self-registration. Organization administrators provision patient and staff users. Cognito handles invitation/activation, password authentication, and MFA enrollment. HealthCloud stores only the identity-provider subject and application identity/authorization data; the HealthCloud database does not store application passwords.

20.3 Final login flow

```
Browser / React
-> Spring Boot BFF login endpoint
-> Cognito managed authentication
-> authorization code returned to Spring Boot
-> Spring Security exchanges code for Cognito tokens
-> access / ID / refresh tokens remain server-side
-> Spring Session JDBC persists server session in PostgreSQL
-> browser receives secure HealthCloud session cookie
```

Spring Boot is the confidential OAuth client. Its client secret is stored in AWS Secrets Manager and never shipped to React or source control. PKCE may be used as defense in depth, but browser-side token storage is not used.

20.4 Session cookie and browser origin

Cookie:

```
HttpOnly = true
Secure = true
SameSite = Lax
Path = /
```

HealthCloud uses one public browser origin. CloudFront routes static React assets to S3 and application routes such as `/api/*`, `/oauth2/*`, `/login/*`, and `/logout` to the Spring Boot application through the ALB. This avoids normal production cross-origin API calls.

20.5 CSRF

Because the browser authenticates with an ambient session cookie, state-changing requests use Spring Security CSRF protection. React receives a non-sensitive CSRF token and returns it in a request header such as `X-CSRF-TOKEN`. The session cookie remains `HttpOnly`.

20.6 Final MFA policy

MFA is required for every interactive HealthCloud user: patient, provider, care coordinator, claims reviewer, organization administrator, platform administrator, and auditor. TOTP authenticator MFA through Cognito is the target method.

Break-glass additionally requires fresh reauthentication and fresh MFA regardless of the existing session.

20.7 Token and session lifetimes

Item	Final policy
Cognito access token	15 minutes
Cognito ID token	15 minutes
Cognito refresh token	8 hours
HealthCloud idle session timeout	30 minutes
HealthCloud absolute session lifetime	8 hours
Remember-me	Not supported

20.8 Identity versus application authorization

Cognito proves identity. HealthCloud PostgreSQL determines organization membership, roles, permissions, patient/provider/care-team relationships, request/claim assignments, consent, purpose, business need, field visibility, and break-glass status.

21. Authorization Architecture

21.1 Final hybrid model

HealthCloud uses broad role permissions plus attribute/relationship policies. Authorization layers:

1. authenticated session;
2. account ACTIVE state;
3. tenant authorization;
4. function-level permission;
5. object-level authorization;
6. patient/provider/care-team/request/claim relationship;
7. consent and purpose evaluation;
8. business-need rule;
9. field-level authorization/masking;
10. audit generation.

21.2 Simplified permission matrix

Clinical and workflow permissions

Capability	Patient	Provider	Coordinator	Claims reviewer
Own patient data	Yes	-	-	-
Assigned patient data	-	Conditional	Conditional	Claim-relevant only
Create request	Yes	Conditional	Yes	Claim-related
Assign/triage request	No	Limited	Yes	Claim queue only
Review claim	Own result	Supporting role	Coordination role	Yes
Break-glass initiation	No	Yes	Yes	No

Administrative and oversight permissions

Capability	Organization administrator	Platform administrator	Auditor
Patient data	Controlled, purpose-limited	No casual access; exceptional incident only	Metadata only unless separately authorized
Manage organization users/facilities	Yes, own organization	Tenant lifecycle/global technical scope	No
Cross-tenant patient access	No	Exceptional incident only, reason + audit	No
Claims oversight	Organization oversight	No ordinary claim access	Audit only
Privacy review	Permission-based	No ordinary privacy-review role	Permission-based
Break-glass initiation	No	No	No

21.3 Access-decision process

```

Authenticate session
-> Resolve app user, tenant, roles, permissions
-> Load resource within tenant scope
-> Check function permission
-> Check object/relationship
-> Evaluate consent + purpose
-> Evaluate business need
-> Apply field visibility
-> Record allow/deny decision
-> Return only permitted data or secure denial

```

21.4 Purpose-of-use source

Purpose is not arbitrary browser input. The backend defines approved purposes for each action. Examples:

```

Clinical context -> CARE_COORDINATION
Claims review -> CLAIM_PROCESSING
Document inspection -> DOCUMENT_REVIEW
Appointment workflow -> APPOINTMENT_SUPPORT
Benefit workflow -> BENEFIT_SUPPORT

```

If multiple purposes are legitimate, the UI presents an allowlist and the backend validates the selected value.

21.5 Secure disclosure behavior

For unauthorized object reads where revealing existence would itself disclose information, return a tenant-safe 404. Internally record `AUTHORIZATION_DENIED`.

Use 403 when the authenticated user legitimately knows the resource exists but lacks permission for an action, such as an invalid state-change permission.

21.6 Database defense

Tenant isolation is enforced by application policy, tenant-scoped queries, organization-aware foreign-key relationships/constraints, and negative tests. PostgreSQL Row-Level Security is intentionally not part of the initial implementation because duplicating the application policy model with pooled-session tenant variables would add complexity. RLS remains a future defense-in-depth option.

22. Consent Management

22.1 Consent directive model

Each active consent directive includes:

```

effect = GRANT | DENY
purpose

```

```

dataCategory
providerScope
careTeamScope
organizationScope
effectiveFrom
effectiveTo
status
version

```

22.2 Consent purposes

```

CARE_COORDINATION
CLAIM_PROCESSING
DOCUMENT_REVIEW
APPOINTMENT_SUPPORT
BENEFIT_SUPPORT

```

22.3 Consent-controlled data categories

```

DEMOGRAPHICS_CONTACT
CARE_COORDINATION
CLINICAL_CONTEXT
CLAIMS_BENEFITS
DOCUMENTS

```

These are not controlled by patient consent because they are required for platform integrity/governance:

```

SECURITY_METADATA
AUDIT_METADATA
PLATFORM_OPERATIONAL_DATA

```

22.4 Consent states and versioning

```

SCHEDULED
ACTIVE
REVOKED
EXPIRED
SUPERSEDED

```

Activated consent is immutable. A modification creates a new version and marks the previous version **SUPERSEDED**. Revocation changes later authorization decisions immediately but never deletes prior history.

22.5 Conflict evaluation

1. Find applicable **ACTIVE** directives for actor, organization, patient, purpose, data category, and current time.
2. Use the highest specificity:

```

Provider-specific
> Care-team-specific
> Organization-wide

```

3. When equally specific applicable directives conflict, **DENY** wins.
4. If no applicable **GRANT** exists, deny by default.

Break-glass is a separate emergency authorization mechanism and never creates or rewrites consent.

22.6 Consent audit and notifications

Grant, deny, supersession, revocation, expiration handling, and authorization use are audited. Consent lifecycle events may create in-app/email notifications with minimum necessary content.

23. Field-Level Data Visibility and Masking

23.1 Classification model

Approved classification labels:

```
INTERNAL
CONFIDENTIAL
SENSITIVE_HEALTH_DATA
AUDIT_ONLY
SECURITY_SECRET
```

Field mapping is implemented in Phase 0/3 design artifacts and code, using the approved categories and policies.

23.2 Visibility rules

Visibility can depend on role, tenant, relationship, consent, purpose, business need, resource state, and export context.

Examples:

- Claims reviewer receives claim-relevant clinical fields, not unrelated treatment details.
- Assigned provider receives permitted clinical context for care coordination.
- Auditor receives audit metadata without ordinary medical content.
- Coordinator receives information required to coordinate the request.
- Platform administrator receives health/tenant operational data without casual patient content.

23.3 Masking/omission

Sensitive fields are omitted, masked, or returned only to authorized callers. The backend creates field-safe DTOs; the frontend is not trusted to hide forbidden values after receiving them.

23.4 Exports/logs/events

Exported data follows the same authorization and masking rules. Fields omitted from an API response must not reappear in logs, traces, notification bodies, or Kafka events.

24. Emergency Break-Glass Access

24.1 Purpose and eligibility

Break-glass is used only for a genuine emergency when normal access is insufficient. Only PROVIDER and CARE_COORDINATOR may initiate it.

24.2 Preconditions

- same organization as the patient;
- ACTIVE account;
- normal access failure or insufficient normal relationship;
- mandatory justification;
- fresh reauthentication;
- fresh MFA.

24.3 Session limits

- Duration: 15 minutes.
- No extension; new emergency requires a new session.
- Only one active break-glass session per user + patient.
- No general bulk export.

24.4 Permitted emergency data

- patient identity;
- permitted contact information;
- Limited Clinical Context;
- relevant encounter/treatment summaries;
- active care-related information.

Full claim financial history, unrelated documents, audit records, other patients, and general export are not automatically available.

24.5 Notification and review

Organization administrator and a `PRIVACY_REVIEW` holder are notified immediately. Review must be completed within 24 hours, and the initiator cannot review the same break-glass event.

24.6 Break-glass states

```
REQUESTED
DENIED
ACTIVE
EXPIRED
REVOKED
REVIEW_PENDING
REVIEWED
```

24.7 Audit and abuse detection

High-priority audit captures actor, patient, organization, reason, resources, start/end, review outcome, and correlation information. Security monitoring detects unusual frequency, volume, duration, or repeated emergency access.

25. Security Considerations and Threat Model

25.1 Security principles

- Synthetic data only.
- Least privilege.
- Deny by default.
- Backend enforcement.
- Tenant isolation.
- Minimum-necessary data.
- Defense in depth.
- Secure failure behavior.
- Tamper-evident auditability.

25.2 Data protection

- HTTPS/TLS only.
- RDS encryption at rest enabled.
- Private S3 document bucket with SSE-KMS.
- AWS Secrets Manager for database credentials, OAuth client secret, audit-integrity key, and other secrets.
- No credentials or tokens in source control.
- Custom application-level field encryption is deferred because all project data is synthetic; it is documented as a future enterprise control.

25.3 Browser security

- Single-origin production architecture.
- Strict CORS allowlist; no wildcard on protected APIs.
- Spring Security CSRF protection for state-changing session-authenticated requests.
- Content Security Policy.
- React default escaping.
- No uncontrolled `dangerouslySetInnerHTML`.
- No OAuth tokens in browser storage.
- Secure HttpOnly cookie.

25.4 Main threats and controls

Threat	Final controls
Cross-tenant data access	Backend tenant context, tenant-scoped queries, composite org constraints, negative tests
IDOR/object access	Object and relationship authorization, secure disclosure behavior
Consent bypass	Central consent evaluator, purpose mapping, default deny
Privilege escalation	Role/permission administration, MFA, audit, separation of duties
Token theft	BFF/server-side tokens, secure cookie, short token lifetimes, no token logging
CSRF	Spring Security CSRF token/header validation
XSS	CSP, React escaping, no untrusted HTML, dependency scanning
Sensitive log leakage	Structured allowlisted logging and redaction
Malicious document	File type/size/signature validation, malware scan, quarantine
Audit tampering	Append-only audit role plus per-org HMAC chain
Lost Kafka event	Transactional outbox
Duplicate event	Idempotent consumer store
Invalid workflow update	State machine and optimistic locking
Financial double counting	Transactional accumulator locks and reprocessing delta
Break-glass abuse	Fresh MFA, reason, 15-minute limit, notification, independent review
Injection	Parameterized persistence, validation, secure ORM/repositories
Excessive export	Role/tenant scope, masking, audit, no general bulk export for most roles

25.5 Rate limiting

Do not introduce Redis solely for rate limiting. The production-style AWS design uses AWS WAF rate-based controls for API abuse where required. Exact thresholds are set after Phase 10 load testing rather than invented in advance.

25.6 Security-scanning products

- GitHub CodeQL - Java and JavaScript/TypeScript source analysis.
- Dependabot - Maven, npm, and GitHub Actions dependency updates/security PRs.
- Trivy - container images, dependencies, Dockerfile/IaC misconfiguration, repository secret detection.
- OWASP ZAP - baseline passive scan in CI/staging; controlled active/API scans only against non-production synthetic environments.

Do not initially add overlapping SonarQube, Snyk, Checkov, or Gitleaks unless a later need justifies them.

25.7 Security pipeline policy

- Detected committed secret -> fail.
- New fixable Critical/High security issue -> fail unless explicitly time-bounded and documented.
- Temporary exceptions require owner, reason, expiration, and tracking issue.

26. Secure Logging Rules

26.1 Never log

- passwords;
- OAuth access/ID/refresh tokens;
- authorization headers;
- session secrets;
- MFA secrets;
- client secrets;
- document contents;
- complete clinical text;
- sensitive patient fields;
- raw claim attachments;
- unredacted request bodies containing sensitive data.

26.2 Log instead

- timestamp;
- service/environment;
- correlation ID;
- trace ID;
- safe actor identifier;
- organization identifier where appropriate;
- action;
- opaque resource identifier/type;

- result;
- error/reason code;
- duration.

26.3 Request-body debugging

Sensitive request bodies are not logged by default. Any temporary debugging requires controlled redaction, limited environment scope, and explicit removal after investigation.

26.4 Privacy rationale

Tokens can enable impersonation, passwords may be reused, documents can contain sensitive information, and operational logs often have broader access/retention than application data. Therefore sensitive content must not leak into observability systems.

27. Audit Logging and Explainability

27.1 Audited actions

- patient/claim/document views;
- consent grant/deny/supersession/revocation;
- provider/coordinator assignments;
- request and claim status changes;
- sensitive allowed/denied access;
- role/membership/facility administration;
- claim rules and final decisions;
- manual review and override;
- appeals/reprocessing;
- exports;
- break-glass access and review.

27.2 Audit metadata

Audit records include, as applicable:

```
event_id
organization_id
actor_user_id
actor_role
patient_id
resource_type
resource_id
action
purpose
consent_result
authorization_result
reason_code
reason_text
changed_fields
before_version
after_version
request_id
correlation_id
trace_id
session_reference
emergency_access
```

```

occurred_at
sequence_number
previous_hash
event_hash
key_version

```

27.3 Before/after-value policy

Do not copy full sensitive objects into general audit records. Store changed field names, safe status/configuration values, and before/after version references. Historical sensitive values remain in protected version/history tables.

Safe examples include status: SUBMITTED -> TRIAGED, priority: NORMAL -> HIGH, non-sensitive role codes, reason codes, and version numbers.

Never copy diagnosis text, treatment summaries, contact details, document contents, passwords, tokens, or raw attachments into audit payloads.

27.4 Tamper-evident audit

Use HMAC-SHA-256 with an integrity key held in Secrets Manager:

```
event_hash = HMAC(audit_integrity_key, previous_event_hash + canonical_event_data)
```

Chains are separated by organization, with a separate platform chain for platform-only administrative events. Store sequence_number, previous_hash, event_hash, and key_version. Key rotation preserves old keys for historical verification.

The normal application database role must not have unrestricted update/delete permissions on audit-event rows.

27.5 Explainability

Claims preserve rule-by-rule results, calculations, reason codes, and human-readable explanations. Authorization decisions preserve tenant, role, relationship, consent, purpose, business-need, field-policy result, and grant/deny reason where safe.

28. Data Retention, Deletion, and Legal History

28.1 Modeled production-style retention policy

Data	Final modeled retention
Claims after final resolution	7 years
Adjudication versions	7 years
Appeals/reprocessing	7 years
Consent history after expiration/revocation/supersession	7 years
Audit events	7 years
Care-coordination requests/comments/history after closure	3 years
Clinical summaries linked to claim	Same 7-year claim period
Clinical summaries not linked to claim	3 years
Claim documents	7 years
Request-only documents	3 years
Rejected/quarantined uploads	30 days

Data	Final modeled retention
General application logs/traces	30 days
Security operational logs	90 days
Disabled account profile	1 year, then minimize where possible

These are HealthCloud design policies for the modeled system, not universal legal claims.

28.2 Synthetic environment lifecycle

All actual HealthCloud environments contain synthetic data and may be reset, destroyed, and reseeded regardless of the modeled retention fields. When a writable public/demo environment is kept running, reset the deterministic demo dataset nightly so recruiter/demo activity does not corrupt standard scenarios.

28.3 Preservation rules

- Original claim/adjudication decisions are never overwritten.
- Consent history is preserved.
- Request/claim status histories are preserved.
- Audit attribution remains even after account-data minimization.
- Soft deletion is used where history must remain.
- Hard deletion is allowed only when no preserved history or modeled retention requirement applies.

Part VI - Frontend, Backend, Data, APIs, and Architecture

29. Exact Technology Stack

29.1 Final backend and infrastructure baseline

Technology	Final baseline
Java	25.0.4 LTS
Spring Boot	4.1.0
Maven	3.9.16 with Maven Wrapper
PostgreSQL	17.10
Flyway	12.4.0
Terraform	1.15.8

Spring Security, Spring Data JPA/Hibernate, Jackson, Micrometer, Spring Kafka, JUnit, and compatible supporting Spring dependencies follow the Spring Boot 4.1.0 dependency-management baseline unless a documented compatibility reason requires otherwise.

29.2 Final frontend baseline

Technology	Final baseline
Node.js	24.18.0 LTS
React	19.2.7
TypeScript	6.0.2
Vite	8.1.5
React Router	7.18.1
TanStack Query	5.101.4
React Hook Form	7.82.0
Zod	4.4.3
Material UI	9.2.0

TypeScript 6.0.2 is deliberately chosen over the newer TypeScript 7 line for ecosystem/tooling stability during the project. React Router 7 is deliberately retained rather than immediately adopting a newer major router version.

29.3 Platform and cloud stack

- Authentication: Amazon Cognito / OIDC authorization-code flow with Spring Boot BFF.
- Session: Spring Session JDBC / PostgreSQL.
- Authorization: application-level hybrid RBAC/attribute policy.
- Messaging: Apache Kafka; Amazon MSK Serverless for full AWS validation.
- Documents: Amazon S3.
- Malware scanning: GuardDuty Malware Protection for S3 in AWS validation.
- Frontend hosting: S3 + CloudFront.
- Backend hosting: ECS Fargate behind ALB.
- Database hosting: RDS PostgreSQL.

- Secrets: AWS Secrets Manager.
- CI/CD: GitHub Actions.
- Observability: Spring Boot Actuator, Micrometer, OpenTelemetry, CloudWatch Logs/Metrics, AWS X-Ray/OpenTelemetry traces.
- API documentation: OpenAPI.
- Containers: Docker.

29.4 Testing and security tools

Backend:

- JUnit 5;
- Spring Boot Test;
- Mockito;
- AssertJ;
- Testcontainers;
- MockMvc.

Frontend:

- Vitest;
- React Testing Library.

End-to-end/accessibility:

- Playwright;
- axe-core;
- WCAG 2.2 AA target.

Performance/resilience:

- k6.

Security:

- CodeQL;
- Dependabot;
- Trivy;
- OWASP ZAP.

29.5 Dependency-upgrade policy

The versions above are the initial frozen baseline. Patch/minor/security updates are accepted through lockfiles, Dependabot, CI, tests, and review. Major-version upgrades require deliberate review and an ADR/update when they materially affect the design. The architecture document is not rewritten for every patch release.

30. Frontend Design and Architecture

30.1 Objective

Make HealthCloud easy to understand and operate while ensuring the frontend never becomes the security enforcement layer.

30.2 Frontend architecture

- React/TypeScript application shell.
- React Router 7 feature routing.
- TanStack Query for server state/caching.
- React local state for small UI-owned state.
- React Hook Form + Zod for forms and validation.
- Small session/authentication context.
- Material UI component foundation.
- Typed API client aligned with OpenAPI.
- Shared loading, empty, error, denial, confirmation, and accessibility patterns.
- No Redux initially; add only if complex client-owned state is proven.

30.3 Single-origin and authentication integration

React and API share one public origin through CloudFront routing. The browser carries only a secure HttpOnly HealthCloud session cookie. State-changing requests include the CSRF token header. The frontend never stores Cognito access/refresh tokens.

30.4 Authorization-aware UI

The UI may hide unavailable actions, disable invalid transitions, show masked fields, and explain why an action is unavailable. Backend authorization is still executed for every protected API operation.

30.5 Role-specific screens

Patient

- dashboard;
- own profile summary;
- request list/detail/create;
- consent history/editor;
- document upload/status;
- patient-facing claim-support outcome.

Provider

- assigned-patient list;
- assigned requests;
- allowed limited clinical context;
- response/comment/document panel.

Care coordinator

- request queue;

- triage and assignment;
- priority/due-date/SLA views;
- timeline/comments.

Claims reviewer

- claim queue;
- claim and line detail;
- rule/financial explanation;
- manual review;
- appeal/reprocessing history.

Organization administrator

- organization users/roles;
- facilities and memberships;
- provider/care-team assignments;
- organization settings;
- authorized audit/security review.

Platform administrator

- organization lifecycle;
- platform health;
- global technical configuration;
- privileged incident workflow.

Auditor

- audit search/filter;
- decision history;
- break-glass review;
- export history.

30.6 Important reusable components

- secure data table;
- status badge;
- masked-field display;
- timeline;
- decision explanation panel;
- authorization-denial panel;
- consent summary;
- assignment selector;
- document scan/security status;

- audit filters;
- sensitive-action confirmation dialog.

30.7 Accessibility

- semantic HTML;
- keyboard navigation;
- accessible labels/errors;
- focus management;
- screen-reader support;
- non-color-only status communication;
- responsive design;
- automated axe checks plus Playwright/manual review;
- WCAG 2.2 AA target.

31. Backend Design and Architecture

31.1 Modular-monolith-first decision

The backend is a modular Spring Boot application before any microservice extraction. This supports strong transactions, lower deployment cost, easier local development, and a coherent single-developer implementation while preserving domain boundaries.

31.2 Deployable components

1. React frontend.
2. Spring Boot BFF/API application.
3. Event/outbox worker process/service where asynchronous execution is required.
4. PostgreSQL.
5. Kafka.
6. S3 document storage.
7. Cognito.
8. AWS monitoring/security services.

31.3 Backend modules

- identity and membership;
- organizations/facilities;
- patients/providers/care teams;
- consent;
- authorization policies;
- service requests/workflow;
- clinical context;
- claims intake;

- adjudication;
- manual review/appeals;
- documents;
- audit;
- events/notifications;
- administration;
- reporting.

31.4 Core services/components

- `AuthenticatedUserContext` / session resolver;
- tenant guard;
- authorization policy service;
- consent evaluator;
- field-visibility policy;
- request state machine;
- claim workflow service;
- adjudication engine and rule components;
- document authorization/service;
- audit event builder/integrity component;
- outbox publisher;
- event consumers;
- notification service;
- global error mapper.

Exact class/package names are implementation details; the responsibilities are final.

31.5 Business logic placement

Controllers remain thin. Business rules, authorization, state transitions, claim calculations, audit creation, and event creation live in dedicated service/domain components. Repository access is tenant-safe by design.

31.6 Transactions

For important state changes, one PostgreSQL transaction should atomically perform:

```
Domain record change
+ status/version history
+ audit event
+ outbox event
```

Kafka publication occurs only after the transaction commits.

31.7 Concurrency

Default PostgreSQL isolation is `READ COMMITTED`.

Use optimistic locking/version columns for requests, claims, consent, assignments, prior authorizations, and appeals.

Use row-level locking (`SELECT ... FOR UPDATE` or equivalent) for deductible/coverage accumulator updates where concurrent financial correctness requires it.

31.8 Idempotency

Important retrievable commands require Idempotency-Key:

- create request;
- submit claim;
- start adjudication;
- submit appeal;
- create document-upload authorization.

Store actor, endpoint, key, request hash, response reference, create/expiry time. Retention is 24 hours. Same key/same request returns the original result; same key/different request returns 409 `Conflict`.

32. Database Design

32.1 Strategy

- PostgreSQL 17.10.
- Shared schema/database multi-tenancy.
- Organization-aware relational constraints.
- Flyway 12.4.0 migrations.
- UUID v4 identifiers for externally visible primary IDs.
- Append-only/versioned histories where required.
- S3 for binary documents.
- No PostgreSQL RLS initially.

32.2 Identifier, time, and money rules

Identifiers: UUID v4; sequential IDs are not exposed in public URLs.

Timestamps: UTC, Java `Instant`, PostgreSQL `TIMESTAMPZ`.

Business dates: Java `LocalDate` for date of service, coverage/consent dates, and birth date if represented.

Each organization has an IANA time zone such as `America/New_York` for presentation.

Money: `BigDecimal` / `NUMERIC(19,2)`, USD, `HALF_UP` rounding.

32.3 Core identity/organization entities

- organization
- facility
- app_user
- organization_membership
- facility_membership

- role
- user_role

32.4 Patient/care entities

- patient
- provider
- provider_patient_assignment
- care_coordinator_assignment
- optional care-team relationship tables where needed by the approved workflows
- service_request
- request_assignment
- request_comment
- request_status_history

32.5 Consent entities

- consent / current consent version representation;
- consent_history or immutable version records;
- purpose, category, scope, effect, dates, status, version.

32.6 Limited clinical-context entities

- diagnosis_summary
- treatment_summary
- encounter_summary
- code-system reference fields
- claim/document links.

32.7 Claims entities

- insurance_plan
- benefit_rule_set
- benefit_rule_set_version
- eligibility_record
- deductible/coverage accumulators
- claim
- claim_line
- adjudication
- rule_result
- manual_review
- appeal
- prior_authorization
- referral

- provider-network data.

32.8 Document/operations entities

- document
- audit_event
- outbox_event
- processed_event
- notification
- break_glass_session
- access_review
- Spring Session JDBC tables.

32.9 Major entity design summary

Entity	Purpose	Important relationships / constraints	Sensitivity / audit
organization	Tenant boundary	Unique ID/name/status; root for tenant data	Internal; changes audited
facility	Lightweight location/entity	Belongs to organization	Confidential/internal
app_user	Internal identity mapping	Cognito subject, status	Confidential; admin actions audited
patient	Synthetic patient profile	Exactly one organization	Sensitive health/confidential
provider	Provider profile	Organization/facility membership	Confidential
assignment	Access relationship	Same-org patient/provider; status/version/dates	Sensitive; audited
consent	Patient access directive	Org/patient/effect/purpose/category/scope/version	Sensitive; immutable history
service_request	Care workflow	Org/patient/type/status/owner/version	Sensitive; history/audit
clinical_summary	Limited clinical context	Org/patient/provider/facility	Sensitive health
claim	Claim header	Org/patient/provider/plan/status/version	Sensitive
claim_line	Financial/service item	Belongs to claim; line outcome	Sensitive
adjudication	Immutable decision version	Claim, ruleset, outcome, responsibility	Audit/claim history
document	File metadata	Org/related resource/S3 key/scan state	Sensitive
audit_event	Security/governance history	Per-org chain; append-only	Audit only
outbox_event	Reliable publication	Event type/version/status	Internal

32.10 Structural tenant integrity

For important tenant-owned relationships, organization identifiers participate in unique/composite foreign-key relationships where practical so a programming bug cannot create a NorthCare claim referencing a Green Valley patient. This complements, rather than replaces, authorization and tenant-scoped queries.

32.11 Indexing

Candidate indexes include (organization_id, status), (organization_id, patient_id),

(organization_id,assignee_id,status), (claim_id,version), (organization_id,occurred_at) for audit, (outbox_status,available_at), and assignment lookup indexes. Final indexes are validated through real query plans rather than claimed in advance.

32.12 Migration strategy

Flyway migrations are version-controlled, validated in CI, applied in controlled environment promotion, and designed for safe rolling/reversible or forward-recovery behavior where appropriate.

33. Detailed Claims Data Model

Area	Final design
Claim header	Organization, patient/member, provider, plan, optional facility, type, service period, submission, billed total, workflow status, version
Claim lines	Service/procedure code, diagnosis references, quantity, billed/allowed amounts, line outcome, reasons, responsibilities
Coverage/eligibility	Plan plus effective dates/status
Benefits	Rule-set version, deductible, copay, coinsurance, limits, exclusions
Supporting records	Clinical summaries, documents, referral, prior authorization, network status
Pricing	Allowed amount and rule basis
Adjudication	Immutable version, outcome, patient/plan responsibility, explanation
Manual review	Assigned reviewer, status, notes, resolution/override reason
Appeal	Submitted reason/evidence, state, resolution, reprocessing link
Reprocessing	New adjudication version linked to previous version
Payment	Calculation of responsibility only; no disbursement
Audit	Claim/adjudication/reviewer/actions correlated to request/trace

33.1 Accumulator correctness

Calendar-year deductible/coverage accumulators are updated only by finalized decisions in the same protected transaction. Reprocessing applies a delta from the previous finalized adjudication so responsibility is not double counted.

34. API Design

34.1 Strategy

- API-first design documented with OpenAPI.
- Base path /api/v1.
- Breaking contract change -> /api/v2; compatible additions remain in v1.
- Backend tenant/authorization enforcement.
- Typed request/response DTOs; never serialize unrestricted entities.
- Validation, pagination, sorting, consistent error format, idempotency, and audit.

34.2 Pagination

Use page, size, and sort for ordinary list APIs. Default size 20; maximum 100. Cursor pagination can be adopted only where real behavior proves it necessary.

34.3 Final API groups

- Authentication/session/BFF endpoints.
- Organizations/facilities.
- Users/roles/memberships.
- Patients.
- Provider/coordinator assignments.
- Consents.
- Clinical context.
- Service requests/comments/status/assignment.
- Claims/lines.
- Adjudication/manual review/appeals.
- Documents/upload/download authorization.
- Audit.
- Notifications/preferences.
- Break-glass.
- Reports/exports.
- Administration.

34.4 Previously established representative endpoints

- GET /patients/{id} - protected patient view; tenant/object/field checks.
- POST /requests - create service request.
- PATCH /requests/{id}/status - controlled transition with version/concurrency.
- GET /audit-logs / final v1 equivalent - scoped audit search.

The earlier POST /auth/login application-password endpoint is replaced by the Cognito/BFF OAuth flow and is not an active final API.

34.5 Recommended v1 endpoint patterns

Group	Representative paths	Main policy
Session	/api/v1/me, BFF login/logout/OAuth routes	Authenticated server session
Organizations	/api/v1/organizations, /facilities	Platform/org admin scope
Users	/api/v1/users, /roles, /memberships	Scoped admin permissions
Patients	/api/v1/patients/{id}	Tenant + object relationship + consent/purpose + field policy
Assignments	/patients/{id}/provider-assignments	Coordinator/admin + same-org constraints
Consents	/patients/{id}/consents	Patient/authorized staff + audit
Clinical	/patients/{id}/clinical-context	Relationship + consent + purpose
Requests	/requests, /{id}/status, /comments, /assignments	State machine + role/object policy
Claims	/claims, /{id}, /lines	Claim relationship + minimum necessary fields
Adjudication	/claims/{id}/adjudications	Claims permission + workflow state

Group	Representative paths	Main policy
Reviews/appeals	/claims/{id}/reviews, /appeals	Assigned reviewer/authorized submitter
Documents	/documents/upload-authorizations, /{id}/download-authorization	Related resource authorization
Audit	/audit-events	Auditor/admin scope; metadata masking
Break-glass	/patients/{id}/emergency-access	Provider/coordinator + fresh MFA + reason
Reports	/reports/claims, /reports/workflows, /reports/consents	Tenant/role/masking/export policy

34.6 GET /api/v1/patients/{id} behavior

- Roles: patient, assigned provider, authorized coordinator, claims reviewer with approved claim need, controlled administrator.
- Tenant: resource organization must match authenticated organization unless a separately audited platform incident process applies.
- Relationship: own patient identity, active provider/care/request relationship, or explicitly approved administrative purpose.
- Consent/purpose: required when policy says so.
- Response: only permitted fields.
- Unauthorized existence-sensitive read: secure 404 plus internal denial audit.

34.7 POST /api/v1/requests

- Tenant derived from session.
- Validates request type, patient ownership/relationship, content policy, and creator permission.
- Requires idempotency key.
- Creates request, initial history, audit record, and after Phase 8 an outbox event in one transaction.

34.8 PATCH /api/v1/requests/{id}/status

- Validates tenant, role/assignment, current state, target transition, and resource version.
- Returns conflict for invalid/current-version conflicts.
- Records from/to state, actor, reason, history, audit, and event.

34.9 API precautions

- Never trust browser-supplied organizationId for tenant ownership.
- Never return unrestricted entity graphs.
- Do not expose presigned URLs without fresh authorization.
- Preserve minimum necessary data in errors and response DTOs.
- Audit exports and sensitive reads.
- Separate API and event versioning.

35. Error-Handling Model

35.1 Standard shape

Final OpenAPI defines a consistent structure conceptually containing:

```
{
  "code": "AUTHORIZATION_DENIED",
  "message": "The requested operation is not permitted.",
  "correlationId": "opaque-id",
  "details": []
}
```

35.2 Categories

Error	Behavior
Validation	400 with field-safe details
Unauthenticated session	401
Authorization	403 or secure 404 according to disclosure policy
Consent denial	Authorization denial + audit reason
Tenant violation	Secure denial; security metric/audit
Not found	Tenant-safe 404
Invalid workflow/state conflict	409 or domain-safe error
Optimistic-lock conflict	409; caller reloads current version
Idempotency mismatch	409
Claim-rule business outcome	Explainable domain outcome, not technical 500
Document scan/upload failure	Controlled document state/error
Messaging failure	Business transaction remains; outbox retries
Internal error	Generic 500 with correlation ID, no stack trace to caller

36. Total Project Architecture

36.1 High-level architecture

```
Browser
-> CloudFront single public origin
  -> S3 React static assets
  -> ALB -> ECS Fargate Spring Boot BFF/API
      -> Cognito OAuth/OIDC
      -> RDS PostgreSQL
      -> private S3 documents
      -> outbox table
          -> Kafka/MSK
              -> worker
                  -> notifications / SLA / asynchronous processing

Observability:
Spring Boot Actuator + Micrometer + OpenTelemetry
-> CloudWatch Logs/Metrics + X-Ray/traces

Delivery:
GitHub Actions -> ECR / Terraform -> AWS environments
```

36.2 Security boundaries

- Browser/user device.
- CloudFront/public edge.
- Cognito identity boundary.
- ALB/application boundary.

- ECS application/security context.
- Private RDS data boundary.
- Private S3 document boundary.
- Kafka producer/consumer boundary.
- Operations/admin boundary.

36.3 Local-development architecture

```
React
Spring Boot
PostgreSQL
Apache Kafka (KRaft)
Mailpit
FakeMalwareScanner
Docker Compose
```

Most development occurs locally to avoid unnecessary cloud spend.

36.4 Full AWS validation/demo architecture

```
CloudFront + S3 frontend
Cognito
ALB
ECS Fargate API/BFF
ECS Fargate worker
RDS PostgreSQL
Private S3 documents
GuardDuty S3 malware scanning
MSK Serverless
Secrets Manager
CloudWatch / X-Ray
Terraform
GitHub Actions
```

This profile is used for integration, performance, security, event reliability, failure/recovery, observability, CI/CD, evidence collection, and final demo/video. Expensive/nonessential resources are destroyed after evidence is captured and can be recreated from Terraform when needed.

36.5 Cache/search decision

No Redis, Elasticsearch, or OpenSearch initially. PostgreSQL handles filtering, reporting, full-text/trigram search where appropriate, and pagination. Add specialized infrastructure only after measured need.

37. High-Level and Low-Level Design

37.1 High-Level Design deliverables

- System-context diagram.
- Container architecture.
- AWS deployment diagram.
- Trust-boundary/security diagram.
- Authentication sequence.
- Authorization decision sequence.
- Consent evaluation sequence.
- Request transition sequence.

- Document upload/download sequence.
- Claim adjudication sequence.
- Appeal/reprocessing sequence.
- Break-glass sequence.
- Transactional outbox/Kafka sequence.

37.2 Non-functional qualities

No unsupported numeric scale/SLA is invented. The architecture is designed for:

- tenant isolation;
- security/privacy;
- maintainability;
- transactional correctness;
- explainability;
- recoverability;
- accessibility;
- cost awareness;
- evidence-based scalability and performance.

37.3 Low-Level Design direction

LLD defines modules, domain models, interfaces, state machines, repositories, authorization/consent policy components, claims rules, event handlers, transaction boundaries, error mappings, and DTOs. Exact class names remain implementation-level choices.

38. Status and Workflow State Machines

Aggregate	Final states / behavior
User account	INVITED, ACTIVE, SUSPENDED, DISABLED
Provider/coordinator assignment	PENDING, ACTIVE, EXPIRED, REVOKED
Consent	SCHEDULED, ACTIVE, REVOKED, EXPIRED, SUPERSEDED
Service request	DRAFT, SUBMITTED, TRIAGED, ASSIGNED, UNDER_REVIEW, NEEDS_INFORMATION, APPROVED, REJECTED, CANCELLED, CLOSED
Document	PENDING_UPLOAD, UPLOADED, SCANNING, AVAILABLE, QUARANTINED, SCAN_FAILED, REJECTED, DELETED
Claim workflow	DRAFT, SUBMITTED, VALIDATING, READY_FOR_ADJUDICATION, ADJUDICATING, PENDED_FOR_REVIEW, UNDER_MANUAL_REVIEW, FINALIZED, UNDER_APPEAL, REPROCESSING
Adjudication outcome	APPROVED, PARTIALLY_APPROVED, DENIED, PENDED_FOR_MANUAL_REVIEW
Claim line	APPROVED, DENIED, PENDED
Prior authorization	PENDING, APPROVED, DENIED, CANCELLED, EXPIRED
Manual review	PENDING, IN_REVIEW, NEEDS_INFORMATION, RESOLVED, CANCELLED; resolution APPROVE/DENY/OVERRIDE
Appeal	SUBMITTED, UNDER_REVIEW, NEEDS_INFORMATION, REPROCESSING, RESOLVED, WITHDRAWN; resolution UPHELD/OVERTURNED/PARTIALLY_OVERTURNED
Break-glass	REQUESTED, DENIED, ACTIVE, EXPIRED, REVOKED, REVIEW_PENDING, REVIEWED

Every transition defines authorized initiator, preconditions, tenant/relationship conditions, required reason where applicable, audit event, notification behavior, and concurrency protection.

Part VII - Production Engineering

39. Advanced Production Features

Feature	Problem solved	Design	Failure/verification	Phase
Hybrid authorization	Roles alone over-grant access	Central tenant/relationship /consent/purpose/field policy	Cross-tenant/unassigned/wrong-purpose tests	3
Field visibility	Sensitive overexposure	Backend response shaping	Reviewer requests prohibited clinical field	3
Transactional outbox	DB/Kafka dual-write loss	Domain/history/audit/outbox in one DB transaction	Kafka unavailable after commit	8
Idempotency	Duplicate commands/events	Idempotency-Key + processed-event store	Same command/event retried	1-8
Optimistic locking	Lost concurrent update	Version field	Two users transition same request	2
Retry/backoff	Temporary async failure	Consumer retry policy	Dependency outage	8
DLQ	Poison/repeated failures	Dead-letter topic/queue	Repeated processing failure	8
Tamper-evident audit	Silent audit modification	Per-org HMAC chain	Alter historical row -> verification fails	7
SLA escalation	Overdue workflow	Worker/event scheduling	Due-date threshold crossed	8
Direct S3 upload	Avoid large file API proxy	Short-lived authorized upload	Unauthorized/reused upload blocked	3
File-security checks	Malicious content	validation + malware scan + quarantine	Threat/scan failure	3/8
API/event versioning	Safe evolution	/api/v1, schemaVersion	Old/new compatibility	0 onward
Retention controls	Unbounded sensitive history	Retention metadata/lifecycle	Expired synthetic record policy	7/11

40. Event-Driven Architecture

40.1 Transactional outbox

Important business actions write domain state, history, audit, and outbox records in the same transaction. A publisher asynchronously sends committed outbox events to Kafka. Kafka availability therefore does not control whether the business transaction itself commits.

40.2 Event catalog

Approved event examples:

```

RequestCreated
RequestAssigned
RequestStatusChanged
ConsentGranted
ConsentRevoked
DocumentUploaded
ClaimSubmitted
ClaimAdjudicated
ClaimPended
AppealSubmitted
NotificationRequested

```

40.3 Domain topics

```
healthcloud.workflow.events
healthcloud.consent.events
healthcloud.claims.events
healthcloud.documents.events
healthcloud.notifications.commands
```

Do not create one topic per event type.

40.4 Partition key and ordering

Use:

```
organizationId:aggregateType:aggregateId
```

This preserves useful per-aggregate ordering without claiming global ordering.

40.5 Event envelope

```
eventId
eventType
schemaVersion
occurredAt
organizationId
aggregateType
aggregateId
correlationId
traceId
actorId
payload
```

40.6 Payload privacy

Events contain references and minimum necessary metadata, not complete healthcare records. For example, `ClaimAdjudicated` should carry claim/adjudication identifiers and outcome, not full diagnoses, patient names, documents, or the entire claim. Consumers needing more data retrieve it from the authoritative store under appropriate service authorization.

40.7 Serialization/version compatibility

Use JSON + JSON Schema stored in the repository and validated in CI. Within a major schema version, only backward-compatible optional fields are added; existing meanings are not changed. Breaking changes create a new major `schemaVersion`. Consumers support current and immediately previous major versions during migration.

A separate schema-registry service is not required initially.

40.8 Reliability controls

- idempotent consumers using `(consumer_name, event_id)`;
- retry with backoff;
- dead-letter handling;
- controlled operator-initiated replay by time/event range;
- audit of replay operations;
- duplicate handling;
- correlation/trace propagation;
- consumer lag and outbox backlog monitoring.

40.9 Cloud/local strategy

Local development runs Apache Kafka in Docker Compose using KRaft. Full cloud validation uses MSK Serverless for a limited period. MSK is not left running permanently solely for portfolio availability.

41. Search, Reporting, and Exports

41.1 Search/filtering

- patient/assigned-patient search;
- service-request filters;
- claims filters;
- audit search;
- saved filters;
- pagination/sorting.

PostgreSQL handles initial search/reporting; full-text/trigram indexes may be used when justified.

41.2 Reports

- claims decision report;
- workflow-performance report;
- consent-expiration report;
- security/audit views.

41.3 Export permissions

- Claims reviewer: authorized claim reports within tenant/assignment scope.
- Organization administrator: organization workflow/claim reports with masking.
- Auditor: authorized audit reports.
- Patient/provider/coordinator: no general bulk CSV export.

Every export re-runs authorization, applies field masking, is organization-scoped, and creates an audit event.

42. Notifications

42.1 Channels

Only:

```
IN_APP
EMAIL
```

No SMS, push, WhatsApp, Twilio, or voice notification requirement.

42.2 Notification categories

- consent grant/deny/revocation/expiration;
- assignments;
- request status and information needed;

- SLA escalation;
- claim submission/final outcome/manual-review needs;
- appeal updates;
- break-glass/security alerts;
- document scan/quarantine status;
- important administrative/security events.

42.3 Privacy

Email and notification payloads contain minimum necessary information. Detailed clinical or claim information remains inside the authenticated application.

42.4 Delivery reliability

Notification requests are event-driven, idempotent, retried safely, and auditable. Duplicate event delivery must not send duplicate user notifications.

Part VIII - Cloud, Delivery, Testing, and Operations

43. AWS Cloud Deployment

43.1 Full AWS validation/demo architecture

- Region: us-east-1.
- Frontend: S3 + CloudFront.
- Identity: Cognito.
- Edge/security: CloudFront; AWS WAF rate-based controls where required.
- Backend: ALB -> ECS Fargate Spring Boot BFF/API.
- Worker: ECS Fargate asynchronous worker.
- Database: RDS PostgreSQL 17.10, small Single-AZ demo sizing.
- Documents: private S3 + SSE-KMS + versioning.
- Malware: GuardDuty Malware Protection for S3.
- Events: MSK Serverless during validation.
- Secrets: Secrets Manager.
- Monitoring: CloudWatch + OpenTelemetry/X-Ray.
- Container registry: ECR.
- IaC: Terraform.
- Delivery: GitHub Actions.

43.2 Cost-conscious sizing

Representative demo sizing:

API Fargate:	0.5 vCPU / 1 GB
Worker:	0.25 vCPU / 512 MB
RDS PostgreSQL:	db.t4g.micro, Single-AZ, 20 GB gp3

These are portfolio/demo sizes, not production capacity claims.

43.3 Network/trust boundaries

The portfolio validation environment uses a public ALB with tightly restricted security groups and a private database. Avoid a NAT Gateway solely for the demo if it creates unnecessary idle cost. The production-style architecture documentation may show private application subnets, NAT/VPC endpoints, and Multi-AZ database options as a higher-availability target rather than claiming they are continuously deployed.

43.4 Infrastructure lifecycle

Most development is local. Full AWS infrastructure is provisioned when needed for integration, performance, security, resilience, observability, recovery, and demo evidence. Expensive resources - especially MSK - are destroyed after validation and can be recreated using Terraform.

44. Environment Strategy

Environment	Purpose	Data	Infrastructure	Cost/access principle
Local	Daily development	Deterministic synthetic seed	Docker Compose	No AWS compute requirement
Development	Shared cloud integration when needed	Synthetic	Small cloud resources	Destroy/scale down when idle
Staging	Release/E2E/security/migration validation	Synthetic	Production-like configuration	Temporary/on-demand
Demo	Recruiter/interview proof	Resettable synthetic scenarios	Full or selected validation stack	Run only during active demo periods
Production-style target	Architecture reference only	Synthetic conceptually	HA/private-subnet reference	Not claimed as a real production deployment

Each environment has separate configuration, secrets, names/tags, and Terraform state. A real prod tenant with patient traffic is not part of this portfolio project.

45. CI/CD Plan

45.1 Source-control workflow

- Monorepo.
- Feature branches.
- Pull requests.
- Protected main branch.
- Required CI checks before merge.
- Tagged/approved demo releases.

45.2 Pull-request pipeline

- Maven backend build.
- Backend unit/repository/integration tests as available.
- Frontend lint/typecheck/build/unit tests.
- OpenAPI/contract checks.
- Static analysis.
- CodeQL.
- Dependabot/security dependency evaluation.
- Trivy filesystem/container/IaC/secret checks.
- Flyway migration validation.
- Docker image build validation.

45.3 Main-branch pipeline

- Repeat required quality/security gates.
- Build versioned images.
- Push to ECR.
- Deploy development environment when enabled.

- Apply migrations.
- Run smoke/health verification.

45.4 Staging/demo release

- Deploy already-tested artifact.
- Apply/verify migrations.
- E2E and authorization-negative tests.
- OWASP ZAP against controlled staging/demo.
- Manual approval before demo release.
- Post-deployment smoke test.
- Capture evidence.

45.5 Rollback

- Roll back application artifact.
- Use migration strategy that supports forward fix/recovery when direct rollback is unsafe.
- Restore database only under the documented recovery procedure when required.
- Verify service health and critical workflows after rollback.

46. Testing Strategy

46.1 Test layers

- unit;
- repository;
- API integration;
- authentication/session;
- authorization-negative;
- consent;
- tenant isolation;
- field masking;
- workflow state machines;
- claims rules and calculation;
- Kafka/outbox;
- document security;
- contract/OpenAPI;
- frontend component;
- end-to-end;
- accessibility;
- performance/load/stress;
- resilience/failure;
- security;

- backup/recovery;
- acceptance.

46.2 Representative cases

ID	Test objective
TEN-001	NorthCare user cannot access Green Valley patient
TEN-002	Browser-supplied org ID cannot override session tenant
AUTH-001	Assigned provider receives only permitted fields
AUTH-002	Unassigned provider is denied
AUTH-003	Claims reviewer cannot receive unrelated clinical fields
CON-001	Active GRANT allows approved purpose/category
CON-002	Provider-specific DENY overrides broader grant
CON-003	Revoked/expired directive changes subsequent access
WF-001	Valid request transition succeeds
WF-002	Invalid transition is rejected
WF-003	Concurrent update produces optimistic-lock conflict rather than overwrite
DOC-001	Unauthorized upload/download authorization is denied
DOC-002	Threat/quarantined document cannot become available
AUD-001	Sensitive allowed and denied actions create audit evidence
CLM-001	Multi-line claim calculates deterministic responsibility
CLM-002	Missing required document pends claim
CLM-003	Excluded service produces explained denial
CLM-004	Mixed line outcomes produce partial approval
CLM-005	Appeal/reprocessing creates new adjudication version and accumulator delta
EVT-001	Kafka outage leaves committed event safely in outbox
EVT-002	Duplicate event is processed once by consumer
EVT-003	Repeated failure enters DLQ and can be replayed
BRK-001	Break-glass requires reason + fresh MFA and expires at 15 minutes
BRK-002	Initiator cannot review own break-glass event
EXP-001	CSV export re-runs masking/tenant policy and is audited
REC-001	Backup restore produces verified usable synthetic system

46.3 E2E journeys

At minimum automate and demonstrate:

1. Patient request creation and tracking.
2. Assigned-provider review with allowed and denied comparison.
3. Claims review/adjudication including manual review/partial approval.
4. Administrative/auditor review of the full audit trail.

The final demo additionally shows consent, cross-tenant denial, event processing, observability, and recovery evidence.

46.4 Performance testing

Use k6 profiles for smoke, baseline, load, stress, and recovery. Capture real p50/p95/p99 latency, throughput, error rate, CPU, memory/JVM, database pool usage, query behavior, and event lag. No numeric performance target/result is invented before measurement.

46.5 Coverage expectations

No artificial percentage is required in the design. Critical tenant, authorization, consent, workflow, claims, document, event, and recovery paths must have automated coverage and evidence.

47. Observability and Monitoring

47.1 Application instrumentation

- Spring Boot Actuator.
- Micrometer.
- OpenTelemetry.
- Structured JSON/safe logs.
- Correlation and trace IDs.

47.2 AWS observability

- CloudWatch Logs.
- CloudWatch Metrics.
- AWS X-Ray/OpenTelemetry traces.
- ECS/RDS/MSK/S3 service metrics as relevant.

47.3 Application metrics

- request rate;
- error rate;
- p50/p95/p99 latency after measurement;
- JVM heap/GC;
- CPU/memory;
- database connection pool;
- slow-query evidence where observed.

47.4 Business/workflow metrics

- requests created/triaged/closed;
- overdue/SLA events;
- claims submitted;
- approved/partially approved/denied/pended outcomes;
- manual review counts;
- processing-time evidence after implementation.

47.5 Security metrics

- authorization denials;
- tenant denials;
- consent denials;
- failed authentication/session events where available;
- break-glass sessions;
- privileged administrative actions;
- document-access denials.

47.6 Event metrics

- outbox backlog;
- publisher failures;
- Kafka consumer lag;
- retries;
- DLQ count;
- replay operations;
- notification processing delay.

47.7 Dashboards and alerts

Dashboards:

- service health;
- care/workflow;
- claims;
- security;
- event processing.

Alerts cover high error rate/latency after baselining, ECS restarts, database connection exhaustion, outbox backlog, Kafka lag, DLQ, document-scan failure, abnormal authorization denials, and authentication problems. Exact thresholds are measurement-driven implementation details.

47.8 Evidence rule

Only measured screenshots/results from implemented tests may be used in the portfolio. No fabricated performance, uptime, security, or recovery result is permitted.

48. Backup and Disaster Recovery

48.1 Final design targets

- RPO: 15 minutes or less.
- RTO: 4 hours or less.

These are design targets until Phase 11 measures a real restore exercise.

48.2 Strategy

- RDS automated backup and point-in-time recovery.
- S3 versioning.
- Terraform-based infrastructure recreation.
- Configuration/secrets inventory and recovery procedure.
- Database/document relationship verification after restore.
- Outbox/event reconciliation.

48.3 Restore exercise

1. Declare controlled recovery scenario.
2. Identify affected data/services.
3. Select recovery point.
4. Restore database/recreate infrastructure.
5. Verify S3/document references.
6. Restart API/worker/event flow.
7. Reconcile outbox/Kafka state.
8. Run smoke, tenant, claims, and consistency checks.
9. Record measured recovery time and observed recovery point.

No multi-region active-active capability is claimed unless separately implemented and tested later.

49. Operational Runbooks

Final runbook subjects:

- API/service outage;
- database problem/connection exhaustion;
- Cognito/authentication outage;
- Kafka lag;
- outbox backlog;
- DLQ processing and replay;
- failed deployment;
- failed migration;
- document malware/scan failure;
- abnormal authorization-denial rate;
- suspicious break-glass activity;
- backup restoration;
- secret rotation;
- tenant-isolation incident;
- unauthorized export attempt.

Standard runbook sections: trigger/symptoms, severity, containment, diagnostics, recovery steps, validation, communications, evidence, and follow-up.

50. Cloud-Cost Plan

50.1 Cost philosophy

HealthCloud proves the full production-style architecture without paying to keep expensive infrastructure idle forever. Most development runs locally. The full AWS validation stack is created for controlled testing/evidence and then destroyed using Terraform when not needed.

50.2 Cost-driving services

Billable services include ECS Fargate, ALB, RDS, Secrets Manager, public IPv4/networking, CloudWatch usage, and especially MSK Serverless. Cognito, CloudFront/S3, and other services may be inexpensive or within applicable allowances at portfolio traffic, but the project never assumes the full architecture is permanently free.

50.3 AWS new-account credit strategy

A new AWS account may use available Free Plan credits/allowances to reduce out-of-pocket validation cost. Credits are treated as a prepaid cloud budget, not proof that every service is free. Account/pricing rules must be checked before provisioning because AWS offerings can change.

50.4 Resource lifecycle

- Develop locally.
- Create dev/staging/demo resources only for active validation.
- Use MSK only during event/cloud proof periods.
- Capture logs, metrics, traces, security results, cost data, and video.
- Destroy nonessential resources after evidence collection.
- Recreate from Terraform before an interview/demo if needed.

50.5 Budget guardrails

Use AWS Budgets and cost-anomaly monitoring. Suggested personal guardrails are warning thresholds around \$5, \$10, and \$25, with a critical threshold around \$50 for an active validation period. These are personal cost controls rather than application SLAs.

50.6 Cost evidence

Phase 12 Evidence Pack includes Cost Explorer/budget evidence and a documented explanation of cost trade-offs, especially the decision to retain Kafka technically while avoiding 24/7 MSK idle spend.

Part IX - Design-Before-Coding Deliverables

51. Meaning of Phase 0

51.1 Purpose

Phase 0 finalizes the complete direction before major application coding. It prevents contradictory implementation, makes security and tenant boundaries explicit, and provides contracts for frontend, backend, database, events, testing, and documentation.

Phase 0 does not attempt to pre-decide every class name, database index, UI spacing rule, autoscaling threshold, or monitoring threshold. Those remain implementation-time details when they require measurement.

51.2 Required deliverables

Deliverable	Purpose	Required contents / completion evidence
Project scope	Prevent scope drift	Final boundary, MVP, advanced, future, exclusions, definition of done
User journeys	Drive APIs/UI/tests	Main success/denial/exception paths for all roles
Role-permission matrix	Make security explicit	Function/object/tenant/field/break-glass permissions
Workflow state machines	Prevent invalid transitions	States, transitions, initiators, preconditions, reasons, concurrency
Consent model	Drive access decisions	GRANT/DENY, purposes, categories, scope, lifecycle, conflict rules
Data-classification matrix	Drive masking/logging	Field classifications, visibility, export/log rules
Claims model	Define bounded engine	Header/lines/rules/calculation/outcomes/review/appeals
ER diagram	Validate relational support	Identity, tenants, care, consent, clinical, claims, documents, operations
OpenAPI contract	Frontend/backend agreement	Main resources, DTOs, errors, auth, pagination, idempotency
Event catalog	Async contract	Events, topics, envelope, versions, producers/consumers, privacy
Threat model	Tie threats to controls/tests	Tenant, object, session, consent, document, event, audit threats
HLD	Architecture agreement	Context, containers, trust boundaries, deployment, sequences
ADRs	Preserve decisions/trade-offs	Major architecture decisions and reconsideration triggers
Product backlog	Implementation planning	Epics, stories, dependencies, acceptance criteria
Test plan	Prove requirements	Representative positive/negative cases and phase gates

51.3 Phase 0 documentation structure

```
docs/
|-- project-scope.md
|-- user-journeys.md
|-- role-permission-matrix.md
|-- workflow-state-machines.md
|-- consent-model.md
|-- data-classification.md
|-- architecture/
|-- er-diagram/
|-- openapi/
|-- events/
|-- threat-model/
```

```
| -- adr/
| -- runbooks/
|-- evidence/
```

51.4 Overall verification

Phase 0 is complete when major roles/workflows are unambiguous, the database supports the workflows, OpenAPI can drive both frontend/backend, the threat model maps to controls/tests, and no Category A decision blocks Phase 1.

52. Architecture Decision Records

52.1 ADR structure

Each ADR records title, status, context, alternatives, decision, advantages, disadvantages, consequences, reconsideration trigger, related diagrams, and implementation phase.

52.2 Final ADR list

1. ADR-001 - Modular monolith first.
2. ADR-002 - Shared PostgreSQL multi-tenancy.
3. ADR-003 - Backend-derived organization context plus tenant-aware constraints.
4. ADR-004 - Cognito authorization-code flow with Spring Boot BFF/server session.
5. ADR-005 - Hybrid role/tenant/relationship/consent/purpose/field authorization.
6. ADR-006 - Limited Clinical Context instead of a full EHR.
7. ADR-007 - Bounded synthetic claims-adjudication engine.
8. ADR-008 - PostgreSQL metadata plus private S3 document content.
9. ADR-009 - Transactional outbox with Kafka.
10. ADR-010 - Append-only per-organization HMAC audit chains.
11. ADR-011 - Optimistic locking for workflow aggregates; row locks for financial accumulators.
12. ADR-012 - ECS Fargate target deployment.
13. ADR-013 - Terraform + GitHub Actions delivery.
14. ADR-014 - Immutable/versioned consent and adjudication history.
15. ADR-015 - Local development + on-demand full AWS validation rather than permanent expensive infrastructure.
16. ADR-016 - Single-origin CloudFront routing and CSRF-protected BFF session.
17. ADR-017 - No Redis/OpenSearch/RLS initially without measured need.

52.3 ADR-001 summary

A modular Spring Boot application is preferred over immediate microservices because it reduces distributed complexity, preserves strong transactions, lowers cloud cost, improves local development, and lets one engineer finish an end-to-end system. Service extraction is reconsidered only if independent scaling/release/ownership evidence emerges.

53. Product-Planning Documents

53.1 Mandatory before coding

- final scope/MVP matrix;

- user journeys;
- role-permission matrix;
- state machines;
- consent model;
- data classification/masking matrix;
- initial ER diagram;
- OpenAPI foundation;
- HLD/trust boundaries;
- threat model;
- major ADRs;
- phase backlog and acceptance criteria.

53.2 Mandatory before later phases

- Claim formulas/rule-set detail before Phase 5.
- Network/prior-auth/referral/appeal state details before Phase 6.
- Break-glass, retention, access-review implementation design before Phase 7.
- Kafka schemas/topics/replay strategy before Phase 8.
- Report/export designs before Phase 9.
- AWS topology/release/rollback design before Phase 10.
- Restore procedure/runbooks and measurement plan before Phase 11.

53.3 Portfolio documentation

Architecture diagrams, decision summaries, test/security evidence, performance/recovery evidence, cost evidence, demo script, screenshots, limitations, README, and case-study material are built alongside implementation rather than invented afterward.

Part X - Decision Finalization and Implementation Strategy

54. Finalizing the Design versus Implementing in Phases

54.1 Complete project design

The final design defines what the whole platform will contain: identity, tenancy, care coordination, consent, privacy, clinical boundary, claims, security, events, cloud, testing, operations, and portfolio evidence.

54.2 Phased implementation

Final design does not mean all components are built simultaneously. The approved sequence is intentionally dependency-aware:

```
Foundation
-> Care coordination
-> Consent/privacy/documents
-> Clinical + claims intake
-> Basic adjudication
-> Advanced claims
-> Advanced governance
-> Event reliability
-> Search/reporting/accessibility
-> AWS/CI-CD
-> Observability/recovery/operations
-> Final evidence/portfolio
```

54.3 Simple example

Authentication and tenant isolation must work before claims; basic claim rules must work before appeals; normal authorization must work before break-glass; correct transactions must exist before Kafka; stable application behavior must exist before performance/operational evidence.

54.4 Final view

The architecture is frozen enough to avoid redesign during coding, while implementation-time details remain free to respond to measurements and code reality.

55. Category A Decisions - Finalized Before Coding

Decision	Final result	Blocking status
Project boundary	Care coordination + consent/privacy + limited clinical context + synthetic claims + production engineering	Not blocked
Organization model	Multiple organizations, one active org per normal user/patient	Not blocked
Multi-tenant storage	Shared PostgreSQL with org-aware constraints	Not blocked
Main journeys	Patient/provider/coordinator/reviewer/admin/auditor/break-glass	Not blocked
Roles/permissions	Final hybrid model + privacy-review permission	Not blocked
Service-request workflow	Final state machine including cancellation	Not blocked
Claim workflow/outcome	Separate final workflow and adjudication models	Not blocked

Decision	Final result	Blocking status
Clinical boundary	Limited Clinical Context only	Not blocked
Classification/masking	Final labels and enforcement model	Not blocked
Authentication	Cognito + BFF + server session + CSRF + MFA	Not blocked
Initial database/API/environment direction	Finalized	Not blocked

56. Category B Decisions - Finalized for Their Phases

Topic	Final result	Target phase
Claims financial model	Final formula/order/rounding/accumulators	5
Network/prior authorization/referral	Final approved advanced scope	6
Partial approval	Line-level outcomes + derived claim result	5/6
Appeals/reprocessing	Versioned decisions and accumulator delta	6
Explainability/human review	Rule results, reason codes, override reasons	5/6
Break-glass	Provider/coordinator, fresh MFA, 15 min, 24h review	7
Audit integrity	HMAC-SHA-256 per organization/platform chain	7
Event architecture	Final topics/key/schema/privacy/idempotency/replay	8
Search/reports/exports	Final roles/masking/CSV policy	9
Cloud-cost strategy	Local + on-demand full AWS validation + teardown	10-12
Backup/recovery	RPO <=15m, RTO <=4h target and restore test	11
Evidence pack	Formal Phase 12 deliverable	12

Part XI - Final Implementation Plan

57. Reconciliation of the Phase Plans

Earlier shorter phase groupings were planning drafts. The authoritative implementation plan is the corrected Phase 0 through Phase 12 sequence below. The earlier apparent numbering gap after Phase 8 was a transcription omission; Phase 9 and Phase 10 are included and required.

No phase is deleted or renumbered.

58. Final Main Implementation Phases

Phase 0 - Project Design and Planning

Goal: Freeze the complete design artifacts required to implement safely and consistently.

Decisions required before starting: None; this is the design-start phase. Final project decisions documented in this Source-of-Truth are converted into implementation artifacts.

Scope:

- project scope and MVP/advanced classification;
- organizations/facilities/roles/journeys;
- role-permission matrix;
- service-request and claim state machines;
- consent model and field classification;
- high-level claims model and calculations;
- ER diagram;
- OpenAPI contract;
- event catalog/schemas;
- threat model;
- HLD/trust boundaries;
- ADRs;
- epics/stories/acceptance criteria.

Backend work: Module boundary design, API/service contracts, authentication/session design, policy interfaces, transaction boundaries.

Frontend work: Route/screen map, role-specific UX, forms, error/denial behavior, single-origin/BFF integration plan.

Database work: ER model, tenant keys/constraints, version/history tables, migration conventions.

Security/privacy work: Permission matrix, consent/purpose rules, logging rules, threat model, MFA/CSRF/session design.

Cloud/DevOps work: Environment plan, Terraform architecture outline, CI/CD design, cost guardrails.

Testing work: Test pyramid, representative cases, acceptance criteria, security-negative plan.

Documentation outputs: Phase 0 document set under `docs/` plus ADRs/OpenAPI/events.

Dependencies: None.

Important proof: Every finalized requirement is traceable to a design artifact and test/acceptance path.

Risks: Overdesign; accidentally turning implementation-time details into blockers; missing tenant/security relationships.

Completion criteria: No Category A item blocks Phase 1; API/data/security designs are mutually consistent.

Demo/portfolio evidence: Phase 0 architecture diagrams and ADRs become early case-study material.

Phase 1 - Application Foundation and Multi-Tenant Identity

Goal: Establish the runnable technical foundation and prove identity/session/tenant isolation.

Decisions finalized before starting: Phase 0 artifacts, exact stack baseline, Cognito/BFF/session/MFA, shared-database tenant model.

Backend work:

- Spring Boot 4.1 modular skeleton;
- Cognito OAuth2 client/BFF;
- Spring Session JDBC;
- current-user/session endpoint;
- global validation/error handling;
- tenant/user context;
- health endpoints;
- organization/facility/user/role/membership modules.

Frontend work:

- React/TypeScript/Vite shell;
- routing;
- session-aware navigation;
- protected routes;
- CSRF integration;
- role-aware menus;
- loading/error/denied states.

Database work:

```
organization
facility
app_user
organization_membership
facility_membership
role
user_role
Spring Session tables
```

Flyway migrations and synthetic seed foundation.

Security/privacy:

- mandatory TOTP MFA;
- secure HttpOnly cookie;
- single-origin routing plan;
- no browser OAuth token storage;
- backend tenant context;
- same-org structural constraints.

Cloud/DevOps: Docker Compose, Maven/npm builds, initial GitHub Actions quality pipeline.

Testing: Authentication/session, account state, CSRF, role navigation, tenant isolation, browser-supplied org override denial.

Outputs: Runnable local application, login/logout, tenant-safe identity foundation.

Dependencies: Phase 0.

Important proof: NorthCare user cannot access Green Valley data; browser cannot choose a different tenant.

Risks: Session/OAuth misconfiguration; tenant context leaking into generic repositories.

Completion criteria: MFA login works, session/CSRF works, tenant context is enforced, Docker Compose and CI succeed.

Phase 2 - Core Care-Coordination Workflow

Goal: Build the core business workflow before adding claims complexity.

Backend work: Patient/provider profiles, provider-patient and coordinator assignments, service requests, request assignments/comments/history, final request state machine, cancellation rules, optimistic locking.

Frontend work: Patient/provider/coordinator dashboards, request creation/detail, triage/assignment, comments/timeline, allowed cancellation controls.

Database work: Patient/provider/assignment/request/comment/status-history tables, organization-aware constraints and indexes.

Security/privacy: Tenant/object/relationship authorization; secure 403/404 behavior; audit of sensitive request access and transitions.

Testing: Patient owns own requests; provider only assigned patients; coordinator workflow; invalid cancellation/transition; concurrent updates.

Documentation: Updated state machine, OpenAPI, workflow sequence, evidence notes.

Outputs: End-to-end care-coordination workflow.

Dependencies: Phase 1 identity/tenant model.

Important proof: Valid transitions work; invalid/concurrent transitions do not silently overwrite state.

Risks: State logic spread across controllers; accidental role-only authorization.

Completion criteria: Core request journeys pass E2E tests and audit history is coherent.

Phase 3 - Consent, Authorization, Privacy and Documents

Goal: Implement HealthCloud's strongest differentiator: consent-aware, purpose-aware, field-aware access plus secure documents.

Backend work: Consent lifecycle/versioning/effects, central policy evaluator, field-visibility component, document authorization, direct-S3 upload/download authorization, audit integration.

Frontend work: Consent screens, masked/denied field behavior, document upload/status, authorization-denial UX.

Database work: Consent/version history, document metadata, audit-event foundation, policy reference data.

Security/privacy: GRANT/DENY conflict logic, default deny, purpose allowlists, data categories, secure logging/redaction, file restrictions, malware scan/quarantine, SSE-KMS/Secrets Manager integration design.

Testing: Assigned provider + active grant, provider-specific deny, revoked/expired consent, wrong purpose, cross-tenant access, claims reviewer field restrictions, unauthorized document access, log-redaction tests.

Cloud/DevOps: S3 private-bucket integration for development/staging; fake scanner locally; GuardDuty integration plan.

Documentation: Authorization decision flow, consent evaluation flow, data-classification matrix, document security sequence.

Outputs: Sensitive access is policy-driven and auditable.

Dependencies: Phase 2 relationships/workflows.

Important proof: Two authenticated users with the same role can receive different results because relationship/consent/purpose differ.

Risks: Policy duplication; sensitive data escaping through DTOs/logs/events.

Completion criteria: Every sensitive allowed/denied path creates appropriate audit evidence and no sensitive data appears in operational logs.

Phase 4 - Clinical Context and Basic Claims Intake

Goal: Add only enough clinical and insurance context for meaningful claims without building an EHR.

Backend work: Diagnosis/treatment/encounter summaries, code-system/version model, claims header/lines, plan/eligibility foundations, claim submission/validation/manual-information-request workflow.

Frontend work: Limited clinical summary screens, claim form/line editor, claim details, reviewer queue, supporting records.

Database work: Clinical summary tables, code references, plan/eligibility, claim/claim-line structures, supporting links.

Security/privacy: Clinical field policies, minimum necessary claims-review access, tenant-aware claim relationships.

Testing: Multi-line claim creation, code/version data, clinical-claim links, reviewer restrictions, tenant constraints.

Documentation: Claim intake flow, clinical boundary diagram/specification, updated ER/OpenAPI.

Outputs: Claims can be created, validated, reviewed, and linked to approved clinical context.

Dependencies: Phase 3 authorization/consent/documents.

Important proof: Claim reviewer sees claim-relevant data without receiving unrestricted medical context.

Phase 5 - Basic Synthetic Claims-Adjudication Engine

Goal: Implement deterministic, explainable synthetic claims rules and responsibility calculations.

Backend work: Rule engine, rule-set versioning, eligibility/coverage/allowed amount/limit/deductible/copay/coinsurance/exclusion/document rules, line outcomes, claim outcome derivation, immutable adjudication versions.

Frontend work: Rule results, calculation breakdown, reason codes, human explanation, pending/manual-review status.

Database work: Benefit-rule-set/version, accumulators, adjudication/rule-result tables.

Security/privacy: Claim calculation access policy; no raw sensitive values in logs/events.

Testing: Formula boundaries, rounding, deductible/limit locks, exclusions, missing document, all-approved/all-denied/pending, rule-version reproducibility.

Documentation: Adjudication flow and rule/calculation specification.

Outputs: Explainable basic adjudication.

Dependencies: Phase 4 claims data.

Important proof: For any decision the platform can show which rules ran and how every financial amount was calculated.

Risks: Financial precision/accumulator races; rule changes affecting history.

Completion criteria: Deterministic tests pass and historical decision versions remain reproducible.

Phase 6 - Advanced Claims Processing

Goal: Add network, prior authorization, referral, anomalies, human review, appeals, and reprocessing.

Backend work: Network status, prior-auth/referral rules, anomaly signals/severity, manual review states/resolution, override reasons, appeal/reprocessing workflow, accumulator delta logic.

Frontend work: Network/prior-auth evidence, reviewer warnings, manual-review queue, appeal submission/review, decision-version timeline.

Database work: Network/prior-auth/referral/manual-review/appeal/reprocessing records and version links.

Security/privacy: Reviewer assignment and business-need checks; audit every override/appeal/reprocessing action.

Testing: Missing/expired prior auth, referral mismatch, network effects, high anomaly -> pending, override reason, immutable original decision, appeal line targeting, reprocessing delta.

Documentation: Appeal/reprocessing sequence and advanced claims trade-offs.

Outputs: Defensible advanced synthetic claims workflow.

Dependencies: Stable Phase 5 adjudication.

Important proof: An original decision is never overwritten and anomaly does not equal fraud.

Phase 7 - Advanced Healthcare Security and Governance

Goal: Add advanced emergency, audit-integrity, retention, and access-review controls.

Backend work: Break-glass sessions/policy, fresh-MFA integration, review workflow, per-org HMAC audit chain, access review, retention metadata/jobs where applicable.

Frontend work: Emergency justification flow, limited emergency view, review screens, audit-integrity/status presentation where useful.

Database work: Break-glass session/review, access-review entities, audit sequence/hash/key-version fields, retention metadata.

Security/privacy: 15-minute emergency access, 24-hour independent review, minimum-necessary emergency fields, no export, tamper evidence, key rotation, disabled-account behavior.

Testing: Break-glass eligibility, fresh MFA, expiry, self-review denial, suspicious-use alert path, audit tamper verification, retention/access-review behavior.

Documentation: Break-glass flow, audit-chain design, governance runbook drafts.

Outputs: Advanced healthcare-style governance controls.

Dependencies: Normal Phase 3 authorization must already be correct.

Important proof: Emergency access is exceptional, time-limited, reviewable, and independently auditable.

Phase 8 - Event-Driven Production Features

Goal: Add reliable asynchronous processing without sacrificing database transaction correctness.

Backend/worker work: Outbox publisher, Kafka producers/consumers, approved topics/partition keys, schemas, notification/SLA processing, idempotency store, retry/backoff, DLQ, controlled replay.

Database work: outbox_event, processed_event, notification/retry metadata.

Security/privacy: Minimum-data event payloads; no sensitive clinical/document payload duplication; replay permission/audit.

Testing: Kafka unavailable, outbox backlog, recovery/drain, duplicate event, poison event, retry/DLQ, replay, event version compatibility, correlation propagation.

Cloud/DevOps: Local Kafka/KRaft; MSK Serverless integration for cloud validation.

Documentation: Event catalog, schemas, outbox sequence, failure/recovery evidence plan.

Outputs: Reliable event-driven behavior.

Dependencies: Stable transactions/workflows from earlier phases.

Important proof: A committed business transaction does not lose its event when Kafka is unavailable.

Phase 9 - Search, Reporting and Enterprise Usability

Goal: Improve operational usability while preserving tenant/security policy.

Backend work: Search/filter/report queries, saved filters where implemented, CSV export, tenant/masking enforcement.

Frontend work: Advanced filters, pagination/sorting, reports, export controls, responsive design and accessibility completion.

Database work: Query/index tuning based on real plans; PostgreSQL full-text/trigram support if needed.

Security/privacy: Role-specific exports, tenant scope, field masking, export audit.

Testing: Cross-tenant search, masked reports, unauthorized exports, pagination, accessibility/keyboard/screen-reader tests.

Documentation: Report catalog, export policy, accessibility evidence.

Outputs: Enterprise-style search/reporting/accessibility.

Dependencies: Stable domain/security models.

Important proof: Export behavior is no less restrictive than interactive UI behavior.

Phase 10 - Cloud Deployment and CI/CD

Goal: Provision and deploy the complete application through controlled AWS environments.

Backend/frontend/cloud work: Docker images, S3/CloudFront, Cognito, ALB/ECS Fargate, RDS, S3 documents, GuardDuty scan integration, MSK Serverless, Secrets Manager, ECR, CloudWatch, Terraform.

CI/CD work: PR checks, main-branch artifacts, ECR publish, Terraform plan/apply, Flyway migration, dev/staging/demo promotion, ZAP, approval gate, smoke test, rollback.

Security/privacy: Secrets, TLS, S3/KMS, security groups, WAF where appropriate, no public RDS.

Testing: Environment integration, migration validation, E2E, security-negative, deployment/rollback tests.

Documentation: AWS deployment diagram, trust boundaries, environment/cost plan, CI/CD flow.

Outputs: Reproducible full AWS validation environment.

Dependencies: Stable application and event architecture.

Important proof: The complete environment can be created from IaC, validated, and safely destroyed/recreated.

Risks: Idle managed-service cost; deployment drift; migrations.

Phase 11 - Observability, Recovery and Operations

Goal: Prove that HealthCloud can be diagnosed and recovered rather than merely deployed.

Backend/cloud work: Actuator/Micrometer/OpenTelemetry, dashboards, alerts, log retention, traces, backup/PITR, S3

versioning, runbooks, cost/budget monitoring.

Testing: Controlled API/worker/Kafka/document/auth/database failures, alerting, diagnosis, recovery; backup restore; measured RPO/RTO evidence.

Documentation: Runbooks, restore procedure, dashboard catalog, cost notes, measured recovery evidence.

Outputs: Operational readiness evidence for a synthetic portfolio environment.

Dependencies: Phase 10 AWS environment and Phase 8 event paths.

Important proof: Demonstrate failure -> alert -> diagnosis -> runbook -> recovery with logs/metrics/traces.

Risks: Mistaking target RPO/RTO for measured results; leaving expensive resources running.

Phase 12 - Final Validation and Portfolio Presentation

Goal: Complete the technical audit and transform the implemented system into a defensible engineering case study.

Testing: Full unit/repository/integration/authorization/consent/workflow/claims/event/document/contract/frontend/E2E/performance/resilience/security/accessibility/recovery suite.

Documentation: README, architecture diagrams, ER, API docs, event catalog, threat model, ADRs, CI/CD explanation, testing/observability evidence, deployment/runbooks, limitations, future improvements, trade-offs.

Portfolio outputs: GitHub repository, live/on-demand demo strategy, two-minute video, case study, resume bullets, three LinkedIn posts, portfolio page, sanitized Evidence Pack.

Evidence Pack: Functional, performance, database, Kafka, security, operations, cloud/cost, CI/CD, and recovery evidence with real measured data only.

Dependencies: All prior phases.

Important proof: Claims on the resume/interview are backed by reproducible artifacts and measurements.

Risks: Overclaiming; leaving sensitive/unredacted logs/screenshots; confusing synthetic validation with real production history.

Completion criteria: Final audit passes, evidence is sanitized and complete, documentation matches implementation, and no false production/compliance claim is present.

59. Final Implementation Sequence

```
Phase 0 - Project Design and Planning
Phase 1 - Application Foundation and Multi-Tenant Identity
Phase 2 - Core Care-Coordination Workflow
Phase 3 - Consent, Authorization, Privacy and Documents
Phase 4 - Clinical Context and Basic Claims Intake
Phase 5 - Basic Synthetic Claims-Adjudication Engine
Phase 6 - Advanced Claims Processing
Phase 7 - Advanced Healthcare Security and Governance
Phase 8 - Event-Driven Production Features
Phase 9 - Search, Reporting and Enterprise Usability
Phase 10 - Cloud Deployment and CI/CD
Phase 11 - Observability, Recovery and Operations
Phase 12 - Final Validation and Portfolio Presentation
```

59.1 Parallel work

- Phase 0 artifacts can progress in parallel as long as their final contracts remain consistent.
- Backend/frontend scaffolding can proceed together after OpenAPI/session contracts are clear.
- Tests and documentation are built alongside each phase rather than postponed.
- Terraform modules and CI research can begin early, but final AWS deployment waits for a stable application.
- Instrumentation can begin before Phase 11; full operational proof occurs in Phase 11.

59.2 Work that must not start too early

- Advanced claims before basic adjudication is stable.
- Kafka as a substitute for correct database transactions.
- Break-glass before normal authorization works.
- Reports before tenant/masking policy is stable.
- Performance claims before controlled measurement.
- Permanent expensive cloud infrastructure before there is a validation need.

59.3 Completion gate

A phase is not complete merely because code exists. Functionality, negative/security cases, documentation, test evidence, and required proof must all pass.

60. MVP Acceptance Criteria

ID	Testable criterion
IDN-001	Administrator-provisioned user authenticates through Cognito/BFF and receives secure server session
IDN-002	Every interactive role requires MFA
IDN-003	Suspended/disabled application user cannot use protected APIs
TEN-001	Cross-organization patient/request/claim access is denied
TEN-002	Client-provided tenant value cannot override session organization
TEN-003	Database constraints prevent important cross-tenant relationships
PAT-001	Patient can access only own permitted resources
CAR-001	Provider sees only active assigned patients under policy
CAR-002	Coordinator can triage/assign within organization
WF-001	Valid request transitions succeed
WF-002	Invalid/cancelled/terminal transitions are rejected
WF-003	Concurrent updates do not silently overwrite state
CON-001	Active consent GRANT can permit approved purpose/category
CON-002	More-specific DENY overrides broader grant
CON-003	Revoked/expired consent changes subsequent access
AUT-001	Object/relationship authorization is backend enforced
AUT-002	Field-level policy removes unauthorized fields
DOC-001	Authorized direct-to-S3 upload works with metadata/scan state
DOC-002	Threat/quarantine or unauthorized download remains blocked
AUD-001	Sensitive allowed and denied actions create audit events

ID	Testable criterion
CLM-001	Claim supports multiple lines and required supporting data
ADJ-001	Basic engine follows final rule/calculation order
ADJ-002	Decision includes line/claim outcome, amounts, reasons, explanation
ADJ-003	Mixed approved/denied lines produce partial approval
SEC-001	Passwords/tokens/documents/sensitive fields do not appear in logs
TST-001	Critical tenant/auth/consent/workflow/claims/document tests pass
DEP-001	Local system runs reproducibly with Docker Compose
DOCS-001	Phase documentation/OpenAPI/state diagrams match implementation
DEMO-001	NorthCare/Green Valley isolation scenario is demonstrable
DEMO-002	End-to-end request/claim/audit story is demonstrable

Part XII - Real-World Engineering and Portfolio Presentation

61. Real-World Engineering Depth

HealthCloud demonstrates security beyond login, real workflow rules, multi-tenant data design, consent/purpose policy, reliable events, immutable decisions, concurrency, failure handling, auditability, cloud delivery, observability, recovery, cost trade-offs, and production ownership.

"Scalable" means maintainable modules, correct database design/indexing, pagination, object storage, asynchronous processing where justified, and measured performance - not unsupported claims of millions of users.

62. Engineering Decisions and Trade-Offs

Decision	Alternatives	Final choice and reason	Trade-off / reconsideration
Backend topology	Immediate microservices vs modular monolith	Modular monolith - simpler transactions, cost, local dev, completion	Extract only after scaling/ownership evidence
Tenant storage	DB per tenant vs shared DB	Shared PostgreSQL + org-aware constraints	Larger logical blast radius; DB-per-tenant future
Identity/session	Browser JWT SPA vs BFF	Cognito + Spring BFF/server session	Server session state; stronger token protection
Authorization	RBAC only vs hybrid	Role + tenant + relationship + consent + purpose + field	More policy complexity; necessary for problem
Clinical scope	Full EHR vs no clinical data	Limited Clinical Context	Deliberately incomplete clinical product
Claims	Simple CRUD vs full insurer engine	Bounded synthetic rule engine	Simplified real-world breadth
Documents	DB blobs vs object storage	PostgreSQL metadata + private S3	Distributed object/metadata consistency
Events	Direct Kafka dual write vs outbox	Transactional outbox	Event publication becomes eventual
Audit	Editable log vs append-only chain	Per-org HMAC tamper evidence	Key/verification management
Concurrency	Last-write-wins vs locking	Optimistic for workflows; row locks for accumulators	Client conflict handling/targeted blocking
Search/cache	Redis/OpenSearch immediately	PostgreSQL first	Specialized service only after measurement
Cloud availability	Permanent full stack vs on-demand	Full AWS validation then teardown	Live backend may be on-demand, but evidence remains

63. Repository Structure

Final monorepo direction:

```
healthcloud/
|-- backend/
|   |-- src/
|   |-- migrations/
|   |-- tests/
|-- frontend/
|   |-- src/
|   |-- tests/
|-- worker/
|-- infrastructure/
|   |-- terraform/
|   |-- environments/
|-- api/
|   |-- openapi/
```

```
|-- docs/
|   |-- architecture/
|   |-- er-diagram/
|   |-- events/
|   |-- threat-model/
|   |-- adr/
|   |-- runbooks/
|   |-- evidence/
|-- synthetic-data/
|-- scripts/
|-- .github/workflows/
|-- docker-compose.yml
|-- README.md
```

A deterministic seed/reset script recreates standard NorthCare/Green Valley scenarios.

64. GitHub README Direction

The README is a concise engineering case study and includes:

1. Project overview and problem.
2. Core engineering story.
3. Healthcare/synthetic-data boundary.
4. Architecture and technology stack with rationale.
5. Multi-tenancy.
6. BFF authentication and MFA.
7. Authorization and consent.
8. Care-coordination workflow/state machine.
9. Limited Clinical Context.
10. Claims intake/adjudication/appeals.
11. Documents/security/audit.
12. Database and API design.
13. Transactional outbox/Kafka.
14. Testing/security scans.
15. Observability/recovery.
16. CI/CD and AWS deployment.
17. Synthetic demo organizations/users.
18. Local setup and environment lifecycle.
19. Screenshots/demo video/evidence.
20. ADRs and trade-offs.
21. Known limitations and future improvements.

The README begins with the engineering problem, not a technology shopping list.

65. Architecture and Design Diagrams

The implementation documentation includes, where they can be faithfully generated from final design:

- system context;

- container architecture;
- backend component/module diagram;
- multi-tenant/database boundary diagram;
- AWS deployment;
- security/trust boundaries;
- ER diagram;
- authentication/BFF flow;
- authorization-decision flow;
- consent-evaluation flow;
- care-coordination state/workflow;
- claims-intake flow;
- adjudication calculation flow;
- appeal/reprocessing flow;
- break-glass flow;
- audit/HMAC flow;
- transactional outbox/Kafka flow;
- CI/CD pipeline.

Where an exact visual would require implementation assumptions, the approved textual sequence/specification is retained until the relevant phase produces the final diagram.

66. Resume and Interview Value

66.1 Interview introduction

HealthCloud is a synthetic-data, multi-tenant care-coordination and claims platform. The central challenge is not simply authenticating users; it is enforcing organization, role, patient relationship, consent, purpose, business need, and field-level visibility while preserving an auditable and explainable history of sensitive decisions.

66.2 Interview stories

- Why shared-db multi-tenancy and how cross-tenant relationships are prevented.
- Why BFF/session instead of browser-held tokens.
- RBAC versus attribute/relationship policy.
- Consent GRANT/DENY specificity/default deny.
- Object-level/IDOR prevention.
- State machines and optimistic locking.
- Claim financial formulas and immutable rule-set versions.
- Partial approval and accumulator reprocessing delta.
- Transactional outbox and Kafka failure behavior.
- Idempotency, retry, DLQ, replay.
- Per-organization HMAC audit chains.
- Secure direct-to-S3 uploads and malware handling.

- Why full AWS is on-demand after evidence collection.
- How performance, recovery, and cost claims are measured rather than invented.

66.3 Resume guidance

Use 2-3 high-signal bullets, not a long feature list. Mention the problem, architecture/security depth, event/reliability design, cloud deployment, and only real measured metrics after implementation.

67. LinkedIn Post Plan

Post 1 - Problem, scope, architecture

Why healthcare workflows need role-, object-, relationship-, consent-, purpose-, and field-aware authorization; why HealthCloud is more than a dashboard; architecture and synthetic-data positioning.

Post 2 - Implementation and engineering challenges

BFF authentication, consent engine, masking, claims calculations, transactional outbox/Kafka, audit integrity, important trade-offs.

Post 3 - Completed system and evidence

Two-minute demo, GitHub/case study, cross-tenant denial, partial claim decision, audit/observability proof, measured results, recovery/cost lessons, future improvements.

68. Demo Video Direction

68.1 Objective

Tell a clear engineering story in roughly two minutes rather than showing every screen.

68.2 Recommended demo actors

- NorthCare patient.
- NorthCare assigned provider.
- NorthCare unassigned provider.
- NorthCare coordinator.
- NorthCare claims reviewer.
- NorthCare auditor/organization administrator.
- Green Valley user for isolation proof.

68.3 Final demo story

1. Patient authenticates with MFA.
2. Patient grants a consent directive.
3. Patient creates claim-support request and uploads synthetic document.
4. Coordinator triages/assigns.
5. Assigned provider receives permitted fields.
6. Unassigned provider is denied.

7. Claims reviewer evaluates multi-line claim.
8. Adjudication engine shows allowed amount, deductible, copay, coinsurance, and partial/approved/denied/pended outcome.
9. Optional manual review/appeal/reprocessing shows immutable versioning.
10. Auditor shows complete access/decision history.
11. Cross-organization request is denied.
12. Outbox/Kafka and observability evidence is shown.
13. End with architecture, security, performance/recovery/cost evidence.

68.4 Claims prohibited in presentation

Do not claim certified HIPAA compliance, real patient use, real insurer production equivalence, real payment processing, unsupported scale/uptime/security, or production results without evidence.

69. Future Improvements

- FHIR-style interoperability/resources.
- More advanced consent interoperability/granularity.
- Cross-organization sharing with explicit trust/governance.
- Database-per-tenant enterprise isolation.
- Dedicated privacy officer role for a larger organization.
- Advanced anomaly/audit analytics.
- Application-level field encryption for real sensitive-data deployments.
- Specialized cache/search infrastructure only after measurement.
- Extraction of independent services only after real scaling/ownership evidence.

Part XIII - Verification, Conflicts, and Traceability

70. Decision History

Earlier idea	Issue	Final replacement	Project impact
Patient/provider dashboard framing	Too shallow/CRUD-like	Consent-aware care coordination and claims platform	Stronger system-design story
Single organization	Cannot prove tenant isolation	Multiple synthetic organizations	Real tenant tests/demo
Database per organization	High operational cost/complexity	Shared PostgreSQL + org-aware constraints	Achievable multi-tenancy
Build only MVP	Risk losing complete design	Finalize full design, implement by phases	MVP becomes first milestone
Full clinical record/EHR	Scope explosion	Limited Clinical Context	Keeps claims/care focus
Simple request/claims CRUD	Weak engineering depth	Bounded explainable adjudication	Strong backend depth
Node.js or Java backend	Ambiguous target stack	Java/Spring Boot	Aligns with target roles
Browser-held JWT SPA	Token exposure and duplicated auth behavior	Cognito + Spring BFF/server session	Stronger auth/session model
Patient optional MFA	Inconsistent sensitive-system posture	MFA for all interactive users	Simpler/stronger policy
Request model without cancellation state	Feature/state inconsistency	Final CANCELLED transitions	Coherent workflow
Claim status mixed with outcome	Domain ambiguity	Separate workflow status and adjudication outcome	Clearer data/model/tests
Notification as future-only	Needed by event phase	In-app + email in Phase 8	Approved async scope
Global audit hash chain	Cross-tenant coupling	Per-org + platform HMAC chains	Better isolation/verification
Permanent full AWS stack	High idle cost	Local + on-demand full validation	Cost-efficient evidence
SQS/App Runner alternative permanent profile	Architecture divergence / availability concerns	Keep one full AWS validation architecture; teardown after proof	Cleaner story

71. Rejected or Replaced Ideas

- Generic healthcare dashboard -> replaced by policy/workflow/claims platform.
- Full EHR -> replaced by Limited Clinical Context.
- Real patient data -> synthetic data only.
- Real insurer engine/payment settlement -> bounded synthetic adjudication.
- Immediate microservices -> modular monolith first.
- Database per tenant initially -> shared DB with tenant controls.
- Browser token storage -> BFF/server session.
- Public patient self-registration -> admin provisioning/invitation.
- Role-only authorization -> hybrid policy.
- PostgreSQL RLS initially -> application + DB constraints; RLS future.
- Redis/OpenSearch initially -> PostgreSQL until measured need.
- SMS/push/WhatsApp notifications -> in-app/email only.

- Permanent MSK -> on-demand MSK validation.
- App-managed passwords -> Cognito identity.
- One global audit chain -> per-organization/platform chains.

72. Resolved Decisions

All previously unresolved or Needs Verification items are now resolved. Historical items are retained here only to show that the final design explicitly closed them:

- exact software baseline;
- Cognito/BFF/session/MFA;
- ICD-10-CM/HCPSC synthetic codes;
- consent categories/effects/conflicts;
- auxiliary states;
- partial approval;
- retention durations;
- RPO/RTO;
- event topics/partition/version/privacy;
- notification channels;
- cost-conscious AWS strategy;
- security-scanning products;
- privacy-review permission;
- facilities;
- patient provisioning;
- audit before/after policy;
- request cancellation;
- claim workflow/outcome separation.

No active unresolved-decision appendix is required.

73. Final Verification Notes

- No Needs Verification item remains.
- Implementation-time decisions remain intentionally flexible when they depend on measurement rather than product design.
- Final database indexes, alarm thresholds, Kafka partition count, autoscaling thresholds, UI styling details, exact class/package names, and test-count percentages are not project-design blockers.

74. Complete Heading Traceability Matrix

The approved heading inventory is represented directly or through consolidation below.

Original topic / heading family	Category	Final section(s)	Status	Resolution / note
Document Status and Purpose	Control	1	Included	Final status/version/limitations

Original topic / heading family	Category	Final section(s)	Status	Resolution / note
Final Project Identity and Positioning	Foundation	2	Included	Final name and HIPAA-aligned wording
Problem Statement / Problem It Solves	Foundation	3	Consolidated	Business, technical, privacy, claims, audit
Why Building / Why Valuable / Resume/Recruiter	Foundation	4, 66	Consolidated	Engineering + career value
Final Project Boundary	Scope	5	Included	MVP/advanced/future/exclusions
Final Recommended Project Scope	Scope	6	Included	Seven core scope domains
Healthcare and Compliance Boundaries	Scope	7	Included	Synthetic/clinical/code policies
MVP and Advanced Matrix	Scope	8	Included	Classification retained
Users and Roles	Users	9	Included	All final roles + privacy permission
Organization/Multi-Tenant Model	Tenancy	10	Included	Shared DB + facilities + constraints
Final User Journeys	Journeys	11	Included	Patient/provider/coordinator/claims/admin/auditor/break-glass
Core Features	Functional	12	Included	Final classifications
Additional Features	Functional	13	Included	Advanced/future separated
Patient/Care Coordination	Functional	14	Included	Requests/states/cancellation/SLA
Clinical Context	Functional	15	Included	Limited context only
Claims Intake	Claims	16	Included	Workflow vs outcome separated
Synthetic Adjudication	Claims	17	Included	Final formulas/rules
Advanced Claims	Claims	18	Included	Network/prior auth/referral/appeal
Documents/Communication	Functional	19, 42	Included	S3/security/notifications
Authentication	Security	20	Superseded earlier model	BFF/session is active final model
Authorization	Security	21	Included	Hybrid policy and 403/404
Consent	Security	22	Included	GRANT/DENY and conflict model
Field Visibility/Masking	Security	23	Included	Backend field policy
Break-Glass	Security	24	Included	15 min / 24h review
Security/Threat Model	Security	25	Included	Controls and scans
Secure Logging	Security	26	Included	Prohibitions/redaction
Audit/Explainability	Governance	27	Included	Per-org HMAC chain
Retention/Deletion/History	Governance	28	Included	Modeled retention + synthetic reset
Exact Technology Stack	Architecture	29	Included	Final versions
Frontend Design	Architecture	30	Included	Final libraries/session/accessibility
Backend Design	Architecture	31	Included	Modular monolith/transactions
Database Design	Architecture	32-33	Included	Domains/constraints/claims model
API Design	Architecture	34	Included	v1/OpenAPI/idempotency
Error Handling	Architecture	35	Included	Secure consistent model
Total Architecture	Architecture	36	Included	Local + AWS validation
HLD/LLD	Architecture	37	Included	Required designs/qualities
Status/State Machines	Workflow	38	Included	Final states
Advanced Production Features	Production	39	Included	Reliability/security controls
Event-Driven Architecture	Events	40	Included	Topics/schema/privacy/replay
Search/Reporting/Exports	Usability	41	Included	Role/masking rules
Notifications	Functional	42	Included	In-app/email

Original topic / heading family	Category	Final section(s)	Status	Resolution / note
AWS Deployment	Cloud	43	Included	Full validation architecture
Environment Strategy	Cloud	44	Included	Local/dev/staging/demo/reference
CI/CD	Delivery	45	Included	PR/main/demo/rollback
Testing Strategy	QA	46	Included	Final toolchain and cases
Observability	Operations	47	Included	Actuator/Micrometer/OTel/CloudWatch/X-Ray
Backup/DR	Operations	48	Included	Final RPO/RTO targets
Runbooks	Operations	49	Included	Final scenarios
Cloud Cost	Operations	50	Included	Local + on-demand AWS + teardown
Meaning of Phase 0	Planning	51	Included	Deliverables and gate
ADRs	Planning	52	Included	Final ADR list
Product Planning Docs	Planning	53	Included	Before coding/phase docs
Design vs Phased Implementation	Strategy	54	Consolidated	Full design + dependency order
Category A	Strategy	55	Included	All finalized
Category B	Strategy	56	Included	All finalized by phase
Phase Reconciliation	Phases	57	Included	0-12, gap resolved
Main Implementation Phases	Phases	58	Included	All 13 phases
Final Sequence	Phases	59	Included	Dependencies/gates
MVP Acceptance	Phases	60	Included	Testable identifiers
Real-World Depth	Portfolio	61	Included	Production ownership
Trade-Offs	Portfolio	62	Included	Major decisions
Repository Structure	Portfolio	63	Included	Monorepo direction
README	Portfolio	64	Included	Case-study structure
Diagrams	Portfolio	65	Included	Required visual set
Resume/Interview	Portfolio	66	Included	Truthful story/evidence
LinkedIn	Portfolio	67	Included	Three-post plan
Demo Video	Portfolio	68	Included	Final story
Future Improvements	Portfolio	69	Included	Separated from build
Decision History	Verification	70	Included	Meaningful replacements
Rejected/Replaced	Verification	71	Included	Historical only
Unresolved/Needs Verification	Verification	72-73	Resolved historical record	Active count zero
Heading Traceability	Verification	74	Included	This matrix
Completeness/Consistency Audit	Verification	75	Included	Below
Information Preservation	Verification	76 / Appendix E	Included	Final confirmation

75. Completeness and Consistency Audit

Topic	Present	Finalized	Internal consistency / key result	Section
Identity/positioning	Yes	Yes	Healthcare-inspired, synthetic, HIPAA-aligned not certified	2,7
Scope/MVP/advanced	Yes	Yes	No scope conflict	5-8
Users/roles/journeys	Yes	Yes	MFA and break-glass policies consistent	9-11

Topic	Present	Finalized	Internal consistency / key result	Section
Organizations/facilities/tenancy	Yes	Yes	Shared DB + org-aware constraints	10,32
Authentication	Yes	Yes	BFF/server session is sole active model	20
Authorization	Yes	Yes	Tenant/object/relationship/consent/purpose/field layers	21
Consent/masking	Yes	Yes	GRANT/DENY specificity/default deny	22-23
Break-glass	Yes	Yes	Provider/coordinator, 15m, 24h independent review	24
Security/logging	Yes	Yes	Tokens/documents/sensitive fields excluded from logs	25-26
Audit	Yes	Yes	Per-org HMAC chain + safe before/after policy	27
Retention	Yes	Yes	Modeled policy separated from synthetic reset	28
Care coordination	Yes	Yes	Cancellation/status model coherent	14,38
Clinical context	Yes	Yes	Limited context, not EHR	15
Claims/adjudication	Yes	Yes	Workflow status separate from outcome	16-18
Partial approval	Yes	Yes	Line-level derivation and accumulator rules	18
Documents/notifications	Yes	Yes	Direct S3 + malware + in-app/email	19,42
Frontend/backend	Yes	Yes	Single-origin BFF; modular monolith	30-31
Database	Yes	Yes	Tenant, histories, claims, ops all supported	32-33
APIs/errors	Yes	Yes	OpenAPI/v1/idempotency/secure denial	34-35
Events	Yes	Yes	Outbox/Kafka/idempotency/version/privacy	40
Cloud/environments	Yes	Yes	Local + full on-demand AWS validation	43-44
CI/CD/security scans	Yes	Yes	GitHub Actions + CodeQL/Dependabot/Trivy/ZAP	45
Testing	Yes	Yes	All finalized features covered	46
Observability	Yes	Yes	Metrics/logs/traces/dashboards/alerts aligned	47
Backup/recovery	Yes	Yes	RPO/RTO targets and restore proof	48
Runbooks/cost	Yes	Yes	Operational and cost lifecycle aligned	49-50
Phase 0-12	Yes	Yes	Dependency order and numbering coherent	51,57-59
Portfolio/evidence	Yes	Yes	No invented metrics; formal Evidence Pack	61-68

76. Information Preservation Report

- All major project topics were reviewed.

- All heading-inventory families are accounted for directly, through consolidation, or as superseded/replaced historical decisions.
- Small but meaningful decisions, caveats, edge cases, examples, security precautions, cost notes, workflow conditions, and portfolio evidence requirements have been preserved.
- Genuine repetition has been consolidated without deleting unique meaning.
- Earlier decisions replaced by later approved decisions are not active requirements.
- Finalized requirements, MVP, advanced scope, future improvements, recommendations, and rejected/replaced ideas are separated.
- No unresolved project decision remains.
- No Needs Verification item remains.
- No new production metric, implementation result, user count, compliance achievement, test result, or benchmark is invented.
- No unique meaningful approved project information is intentionally omitted.

Points of Content Need

No additional unclassified project content remains. All approved information has been incorporated into its appropriate section.

Current Finalization Status

Finalization Area	Current Status
Overall project design	Finalized
Project boundary and scope	Finalized
MVP scope	Finalized
Advanced scope	Finalized
Organization and multi-tenant model	Finalized
User roles and journeys	Finalized
Authentication design	Finalized
Authorization model	Finalized
Consent model	Finalized
Privacy and masking rules	Finalized
Clinical-data boundaries	Finalized
Care-coordination workflows	Finalized
Claims design	Finalized
Claims-adjudication design	Finalized
Provider network and prior authorization	Finalized
Appeals and reprocessing	Finalized
Explainability and human review	Finalized
Emergency break-glass access	Finalized
Database design	Finalized
API design	Finalized
Architecture/HLD/LLD direction	Finalized
Security design	Finalized
Audit design	Finalized
Event-driven architecture	Finalized
AWS/cloud architecture	Finalized
Environment strategy	Finalized
CI/CD strategy	Finalized
Testing strategy	Finalized
Observability strategy	Finalized
Backup and disaster recovery	Finalized
Operational strategy	Finalized
Final implementation phases	Finalized
Phase numbering and sequence	Finalized
Portfolio/documentation direction	Finalized
Project-level unresolved decisions	0
Needs Verification items	0
Implementation-blocking decisions	0

Current Finalization Status: The complete HealthCloud project design has been finalized and is ready to proceed according to the approved implementation plan. No known project-design decision remains unresolved or requires verification before implementation begins.

Final Audit Result

Audit Category	Final Result
Project-level unresolved decisions	0
Needs Verification items remaining	0
Architecture decisions blocking implementation	0
Product-policy decisions blocking implementation	0
Security-policy decisions blocking implementation	0
Claims-design decisions blocking implementation	0
Cloud/deployment decisions blocking implementation	0

The final audit verified:

- completeness of the project information;
- heading coverage;
- preservation of unique details;
- duplication consolidation;
- superseded-decision handling;
- internal architectural consistency;
- technology-stack consistency;
- database/workflow consistency;
- API/user-journey consistency;
- authentication and authorization consistency;
- consent and privacy consistency;
- claims-processing consistency;
- security consistency;
- testing consistency;
- CI/CD and deployment consistency;
- observability and operational consistency;
- implementation-phase consistency;
- phase numbering;
- absence of unresolved or Needs Verification items.

Final Audit Result: PASSED

The HealthCloud project design is finalized and approved to proceed with implementation according to the approved Phase 0 through Phase 12 implementation plan.

This statement describes design readiness. It does not mean the application has already been built, used in production, certified, or operated with real healthcare data.

Appendix A - Architecture Decision Records

A.1 ADR-001 - Modular Monolith First

Context: The platform contains tightly related identity, consent, workflow, claims, audit, and outbox transactions and is built by one engineer.

Alternatives: Immediate microservices; modular monolith.

Decision: Spring Boot modular monolith with a separate event worker where useful.

Reasons: Strong transactions, simpler local development, lower deployment/cognitive cost, faster end-to-end delivery, clear domain boundaries without premature distribution.

Disadvantages: Less independent scaling/deployment. Reconsider when measured load, team ownership, or release independence requires extraction.

A.2 ADR-002 - Shared PostgreSQL Multi-Tenancy

Decision: One shared PostgreSQL database with `organization_id`, tenant-scoped queries, organization-aware constraints, and negative tests.

Alternative rejected initially: Database per tenant due provisioning, migration, backup, monitoring, and cost complexity.

A.3 ADR-003 - Backend-Derived Tenant Context

Tenant identity comes from the authenticated HealthCloud session and membership, never arbitrary frontend selection. Database constraints provide additional structural protection.

A.4 ADR-004 - Cognito + Spring BFF

Use Cognito authorization-code authentication handled by a confidential Spring Boot BFF. OAuth tokens remain server-side; browser receives a secure `HttpOnly` session cookie. Single-origin routing and CSRF protection are mandatory.

A.5 ADR-005 - Hybrid Authorization

Role permission is necessary but insufficient. Final policy includes tenant, object relationship, consent, purpose, business need, field classification, and audit.

A.6 ADR-006 - Limited Clinical Context

Include summaries and claim-support context rather than a full EHR.

A.7 ADR-007 - Bounded Synthetic Claims Engine

Implement explainable eligibility/coverage/allowed amount/cost-sharing/limits/exclusions/manual review rather than claiming full insurer breadth.

A.8 ADR-008 - PostgreSQL Metadata + Private S3 Documents

Keep relational authorization/audit metadata in PostgreSQL and binary content in S3.

A.9 ADR-009 - Transactional Outbox + Kafka

Avoid dual-write loss by writing outbox data in the same transaction as business state and publishing asynchronously.

A.10 ADR-010 - Per-Organization HMAC Audit Chains

Use append-only audit records with HMAC-SHA-256 chaining per organization plus a separate platform chain.

A.11 ADR-011 - Concurrency Strategy

Use optimistic versioning for workflow aggregates and row locks for financial accumulators.

A.12 ADR-012 - ECS Fargate Target Deployment

Use containers/Fargate for the production-style Java deployment rather than changing the main backend runtime to serverless solely for cost.

A.13 ADR-013 - Terraform + GitHub Actions

Infrastructure and delivery are reproducible, reviewable, and evidence-producing.

A.14 ADR-014 - Immutable Consent and Claim Decisions

Never overwrite historical consent or adjudication decisions; modifications/reprocessing create new versions.

A.15 ADR-015 - On-Demand Full AWS Validation

Prove the complete managed architecture, collect evidence, then destroy nonessential billable infrastructure. Do not weaken the core technology stack merely to maintain a permanent \$0 deployment.

A.16 ADR-016 - Single-Origin Browser Architecture

CloudFront serves React and proxies application/auth routes to Spring Boot so session cookies, CSRF, and browser security remain straightforward.

A.17 ADR-017 - PostgreSQL First for Cache/Search

No Redis/OpenSearch/RLS initially; adopt only when real measurements/requirements justify added complexity.

Appendix B - Decision History and Replaced Decisions

The following historical changes are retained for traceability, not as active requirements:

1. Dashboard framing -> secure multi-tenant care/claims platform.
2. Single tenant -> multiple organizations.
3. Database-per-tenant -> shared PostgreSQL initial model.
4. Full EHR -> Limited Clinical Context.
5. Node.js/Express alternative -> Java/Spring Boot final backend.
6. Browser JWT/token-storage architecture -> Cognito + BFF/server session.
7. Optional patient MFA -> mandatory MFA for all interactive users.
8. Request state model without cancellation -> explicit CANCELLED state/transitions.
9. Claim status/outcome combined -> separate workflow status and adjudication outcome.
10. Editable sensitive before/after audit values -> field/version references and safe values only.
11. Global audit hash chain -> per-organization/platform chains.
12. Permanent MSK/full cloud -> on-demand validation + Terraform teardown.
13. Notifications treated as future-only -> in-app/email approved in event phase.
14. Public patient registration -> administrator provisioning/invitation.
15. RLS/Redis/OpenSearch considered as possible depth -> intentionally not initial requirements.

Appendix C - Complete Heading Traceability Matrix

The detailed heading-to-section mapping in Section 74 is the authoritative traceability matrix for the source inventory. It confirms every mandatory heading family is directly included, consolidated, or historically superseded. No source heading is silently ignored.

For implementation maintenance, future edits to this document must update Section 74 whenever a major heading family is added, renamed, consolidated, or superseded.

Appendix D - Completeness and Consistency Audit

The detailed audit in Section 75 is the authoritative completeness/consistency matrix. It checks identity, scope, users, tenancy, journeys, authentication, authorization, consent, masking, break-glass, care coordination, clinical context, claims, documents, events, cloud, CI/CD, testing, observability, recovery, cost, phases, and portfolio evidence.

The final audit result is PASSED, with zero unresolved, verification, architecture-blocking, product-policy-blocking, security-policy-blocking, claims-design-blocking, or cloud/deployment-blocking decisions.

Appendix E - Information Preservation Report

- All major project topics were reviewed.
- All heading-inventory items were accounted for.
- Small but meaningful approved details, warnings, caveats, edge cases, examples, architecture reasons, security notes, testing notes, deployment notes, operational notes, portfolio notes, and sequencing notes were preserved.
- Genuine duplication was consolidated.
- Repeated passages containing unique additional meaning were merged without losing that meaning.
- Superseded decisions do not appear as active requirements.
- MVP, advanced, future, recommendation, and rejected/replaced categories remain distinguishable.
- No unresolved decision remains.
- No Needs Verification item remains.
- No new project requirement was introduced during PDF generation.
- No unique meaningful approved project information was intentionally omitted.
- No unclassified approved content remained; therefore the Points of Content Need section states that all content was placed appropriately.
- Current Finalization Status and Final Audit Result are present.
- No implementation, performance, security, uptime, compliance, or production-usage result was invented.

Appendix F - Final Project Audit and Implementation Readiness

F.1 Readiness summary

HealthCloud is approved to proceed from final design into Phase 0 implementation artifacts and then Phase 1 coding. The architecture, product policies, security model, claims model, cloud strategy, and phase order are finalized.

F.2 Project-level audit counters

Category	Result
Unresolved project decisions	0
Needs Verification items	0
Architecture blockers	0
Product-policy blockers	0
Security-policy blockers	0
Claims-design blockers	0
Cloud/deployment blockers	0

F.3 Implementation readiness statement

The project is ready for implementation according to Phase 0 through Phase 12. Normal implementation-time choices may be made when they do not contradict this source-of-truth. Any future project-level change should be documented through an ADR and corresponding updates to requirements, tests, diagrams, traceability, and evidence.